

CÁLCULO NUMÉRICO Y COMPUTACIONAL COMPUTATIONAL NUMERICAL CALCULATION

CAPITULO 7 Ecuaciones Diferenciales Numéricas

Henry R. Moncada

Universidad Nacional del Callao
Facultad de Ingeniería Mecánica y de Energía

17 de septiembre de 2026

Contenido

- 1 Ecuaciones Diferenciales
 - Problemas de Valor Inicial
- 2 Método de Euler
 - Error del método
 - Ejemplos
 - Aplicaciones
 - Comparación con la solución exacta
 - Extensión a sistemas de EDO
 - Ejemplos
- 3 Método de Euler mejorado – Heun
 - Ejemplos:
- 4 Método de Taylor
 - Aplicaciones
 - Implementación general
 - Análisis del error
 - Resumen
- 5 Metodos de Runge–Kutta
 - Ejemplos
 - Error y tamaño de paso
 - Aplicaciones
 - Runge–Kutta–Fehlberg
- 6 Sistema de Ecuacion Diferenciales
 - Problemas de Contorno o Valores de Frontera (PVF)
- 7 Método del Disparo
 - Formulación matemática
 - Integración mediante RK4

Unidad 7

Ecuaciones Diferenciales (ED)

**Métodos numéricos para la solución de
Ecuaciones Diferenciales Ordinarias**

Problemas de Valor Inicial (PVI)

Ecuaciones Diferenciales Ordinarias (EDO)

Una ecuación diferencial es aquella ecuación que contiene diferenciales o derivadas de una o más funciones.

Una **Ecuación Diferencial Ordinaria (EDO)** es una ecuación diferencial que contiene derivadas de una o más funciones respecto de una sola variable independiente.

Ejemplo

$$\frac{dy}{dt} = f(t, y)$$

Las ecuaciones diferenciales permiten modelar:

- Procesos dinámicos.
- Movimiento de partículas y cuerpos.
- Vibraciones mecánicas.
- Transferencia de calor.
- Vaciado de recipientes.
- Reactores químicos.
- Crecimiento poblacional.
- Deflexión de vigas.

¿Por qué utilizar métodos numéricos?

Para algunas ecuaciones diferenciales es posible obtener una solución analítica.

Sin embargo, en numerosos problemas de ingeniería:

- la ecuación diferencial puede ser no lineal;
- las condiciones iniciales pueden ser complicadas;
- existen varias ecuaciones acopladas;
- las propiedades físicas pueden depender de la solución;
- la solución analítica puede ser demasiado compleja;
- puede no existir una solución analítica sencilla.

Por estas razones se utilizan **métodos numéricos** para obtener una aproximación de la solución.

Idea fundamental

Los métodos numéricos transforman una ecuación diferencial continua en un conjunto de operaciones algebraicas realizadas en puntos discretos.

Ecuaciones Diferenciales Ordinarias (EDO)

Una Ecuación Diferencial Ordinaria (EDO) es aquella **ecuación diferencial que contiene derivadas de una o más funciones con respecto a una sola variable independiente.**

Las ecuaciones diferenciales permiten modelar:

- Procesos dinámicos como: vaciado de recipientes, reactores químicos, movimientos amortiguados, desarrollos poblacionales;
- Situaciones estáticas como: deflexión de vigas y problemas geométricos.

$$\frac{d^2y}{d\mathbf{x}^2} - \frac{dy}{d\mathbf{x}} + 7y = e^x$$

↑ ↑
Todas las derivadas son con respecto a una sola variable dependiente

$$\frac{dy}{dx} = \frac{y}{2x + y^3}$$

A veces no hay una solución analítica, por lo que se necesitan aproximaciones numéricas.

Problema de Valor Inicial (PVI)

Consideremos una EDO de primer orden:

$$\frac{dy}{dt} = f(t, y)$$

acompañada de una condición inicial:

$$y(t_0) = y_0$$

El problema completo se denomina:

Problema de Valor Inicial

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0$$

El objetivo es obtener una aproximación de la solución

$$y(t) = \phi(t)$$

en un conjunto de puntos:

$$t_0, t_1, t_2, \dots, t_n.$$

Discretización del dominio

Sea el intervalo:

$$t_0 \leq t \leq t_f.$$

Dividimos el dominio en N subintervalos.

$$h = \frac{t_f - t_0}{N}$$

Los puntos de la malla son:

$$t_i = t_0 + ih, \quad i = 0, 1, \dots, N.$$

Por tanto:

$$t_0, \quad t_1 = t_0 + h, \quad t_2 = t_0 + 2h, \quad \dots, \quad t_N = t_f.$$

Aproximación

$$Y_i \approx y(t_i)$$

La condición inicial proporciona:

$$Y_0 = y_0.$$

Interpretación geométrica

Partimos de:

$$\frac{dy}{dt} = f(t, y).$$

mediante técnicas de cálculo de primitivas se puede obtener la función

$$y(t) = y + C.$$

La derivada representa la pendiente de la solución:

$$m_i = f(t_i, Y_i)$$

En cada punto (t_i, Y_i) conocemos:

- la posición t_i ;
- la aproximación Y_i ;
- la pendiente $f(t_i, Y_i)$.

La idea del método de Euler consiste en:

avanzar una distancia h utilizando la pendiente actual.

Pendiente $\Rightarrow$ incremento $\Rightarrow$ nuevo valor

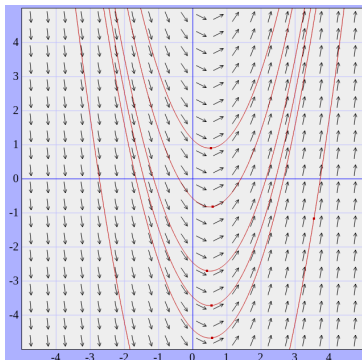
Ejemplo:

$$\frac{dy}{dt} = 2t - 1, \quad y = t^2 - t + C$$

Los métodos numéricos permitirán obtener valores aproximados $Y(t)$ de la solución $\phi(t)$ en puntos t igualmente espaciados.

$$Y_i \approx y(t) = \phi(t)$$

La curva elegida de la familia $\bar{y} + C$ se establece a partir del valor de la función y en el **valor inicial** de la variable independiente $y(t_0)$.



Método de Euler

Partimos de la expansión de Taylor:

$$y(t_{i+1}) = y(t_i) + h y'(t_i) + \frac{h^2}{2!} y''(t_i) + \dots$$

Como:

$$y'(t_i) = f(t_i, y_i),$$

si despreciamos los términos de orden superior:

$$y(t_{i+1}) \approx y(t_i) + hf(t_i, y_i).$$

Por tanto, el método de Euler es:

$$Y_{i+1} = Y_i + hf(t_i, Y_i)$$

Esta es la fórmula fundamental del método.

Método de Euler

El nuevo valor se obtiene utilizando la pendiente calculada en el punto actual.

Derivación mediante diferencias finitas

La derivada puede aproximarse mediante una diferencia hacia adelante:

$$\frac{dy}{dt} \approx \frac{Y_{i+1} - Y_i}{h}.$$

Como:

$$\frac{dy}{dt} = f(t, y),$$

tenemos:

$$\frac{Y_{i+1} - Y_i}{h} = f(t_i, Y_i).$$

Multiplicando por h :

$$Y_{i+1} - Y_i = hf(t_i, Y_i).$$

Finalmente:

$$Y_{i+1} = Y_i + hf(t_i, Y_i)$$

Interpretación

El método de Euler reemplaza la curva real por segmentos rectos que utilizan la pendiente de la solución en cada punto.

Algoritmo del método de Euler

Datos de entrada:

$$f(t, y), \quad t_0, \quad t_f, \quad y_0, \quad N$$

Paso 1:

$$h = \frac{t_f - t_0}{N}$$

Paso 2: inicializar

$$t = t_0, \quad Y = y_0.$$

Paso 3: calcular la pendiente

$$k = f(t, Y).$$

Paso 4: actualizar

$$Y_{\text{nuevo}} = Y + h k.$$

Paso 5: actualizar

$$t_{\text{nuevo}} = t + h.$$

Paso 6: repetir hasta alcanzar t_f .

Pseudocódigo del método de Euler

Algoritmo

Entrada:

$$f(t, y), \quad t_0, \quad t_f, \quad y_0, \quad N$$

$$h = (t_f - t_0) / N$$

$$t = t_0$$

$$Y = y_0$$

Para $i = 0, \dots, N-1$:

$$k = f(t, Y)$$

$$Y = Y + h * k$$

$$t = t + h$$

almacenar(t, Y)

Fin Para

Salida:

$$t_i, Y_i$$

$$Y_{i+1} = Y_i + h f(t_i, Y_i)$$

Algoritmo de Euler diagrama conceptual

$$(t_i, Y_i)$$

↓

$$k_i = f(t_i, Y_i)$$

↓

$$\Delta Y_i = h k_i$$

↓

$$Y_{i+1} = Y_i + \Delta Y_i$$

↓

$$t_{i+1} = t_i + h$$

↓

repetir

Error del método de Euler, Ventajas y limitaciones

La expansión de Taylor es:

$$y(t_{i+1}) = y(t_i) + hy'(t_i) + \frac{h^2}{2}y''(\xi_i)$$

para algún:

$$t_i < \xi_i < t_{i+1}.$$

Euler utiliza:

$$Y_{i+1} = Y_i + hf(t_i, Y_i).$$

Por tanto, el **error local de truncamiento** es:

$$E_{\text{local}} = O(h^2)$$

Mientras que el **error global acumulado** es:

$$E_{\text{global}} = O(h)$$

Conclusión

Euler es un método de **primer orden**. Al reducir h aproximadamente a la mitad, el error global también se reduce aproximadamente a la mitad.

Para problemas que requieren mayor precisión se pueden utilizar métodos como:

Euler mejorado, Heun, Runge-Kutta

Ventajas

- Fácil de implementar.
- Fácil de comprender.
- Requiere pocas operaciones.
- Permite resolver EDO no lineales.
- Es una excelente introducción a métodos numéricos.

Limitaciones

- Precisión relativamente baja.
- Puede requerir pasos pequeños.
- Puede presentar problemas de estabilidad.
- El error aumenta cuando h es grande.

Ejemplo 1: solución de una EDO

Condición inicial: $(t_0, Y(t_0)) = (t_0, Y_0) = (0, 1)$

Paso 1:

$$f(t_0, Y_0) = 2(0) - 1 = -1$$

$$Y_1 = 1 + 0,2(-1) = 0,8$$

$$t_1 = 0,2.$$

Paso 2:

$$f(t_1, Y_1) = 2(0,2) - 0,8 = -0,4$$

$$Y_2 = 0,8 + 0,2(-0,4) = 0,72$$

$$t_2 = 0,4.$$

Paso 3:

$$f(0,4, 0,72) = 0,8 - 0,72 = 0,08$$

$$Y_3 = 0,72 + 0,2(0,08) = 0,736$$

$$t_3 = 0,6$$

Paso 4:

$$f(0,6, 0,736) = 1,2 - 0,736 = 0,464$$

$$Y_4 = 0,736 + 0,2(0,464) = 0,8288$$

$$t_4 = 0,8$$

Paso 5:

$$f(0,8, 0,8288) = 1,6 - 0,8288 = 0,7712$$

$$Y_5 = 0,8288 + 0,2(0,7712) = 0,98304$$

$$t_5 = 1,0$$

Por tanto:

$$y(1) \approx 0,98304$$

Resolver numéricamente el PVI:

$$\frac{dy}{dt} = 2t - y$$

con:

$$y(0) = 1$$

en el intervalo:

$$0 \leq t \leq 1$$

utilizando:

$$N = 5.$$

Por tanto:

$$h = \frac{1 - 0}{5} = 0,2.$$

La función del problema es:

$$f(t, y) = 2t - y.$$

La fórmula de Euler:

$$Y_{i+1} = Y_i + 0,2(2t_i - Y_i)$$

Ejemplo 1: tabla de resultados

i	t_i	Y_i	$f(t_i, Y_i)$
0	0.0	1.00000	-1.00000
1	0.2	0.80000	-0.40000
2	0.4	0.72000	0.08000
3	0.6	0.73600	0.46400
4	0.8	0.82880	0.77120
5	1.0	0.98304	—

La solución aproximada está formada por los puntos:

(t_i, Y_i)
(0, 1)
(0,2, 0,8)
(0,4, 0,72)
(0,6, 0,736)
(0,8, 0,8288)
(1, 0,98304)

Python: Ejemplo 1

```
import numpy as np
import matplotlib.pyplot as plt

def f(t, y):
    return 2*t - y

t0 = 0.0
tf = 1.0
y0 = 1.0
N = 5

h = (tf - t0)/N

t = np.zeros(N + 1)
y = np.zeros(N + 1)

t[0] = t0
y[0] = y0

for i in range(N):
    y[i+1] = y[i] + h*f(t[i], y[i])
    t[i+1] = t[i] + h

print(" t y")
for i in range(N + 1):
    print(f"{t[i]:6.2f} {y[i]:10.6f}")

plt.plot(t, y, 'o-', label='Euler')
plt.xlabel('t')
plt.ylabel('y(t)')
plt.grid()
plt.legend()
plt.show()
```

Ejemplo 2: caída de un cuerpo

Consideremos un cuerpo que cae bajo la acción de la gravedad, despreciando la resistencia del aire.

La velocidad satisface:

$$\frac{dv}{dt} = g$$

con:

$$v(0) = 0.$$

Para:

$$g = 9,81 \text{ m/s}^2$$

y:

$$0 \leq t \leq 5 \text{ s.}$$

Utilizaremos:

$$N = 10$$

por lo que:

$$h = \frac{5 - 0}{10} = 0,5 \text{ s.}$$

La ecuación de Euler es:

$$V_{i+1} = V_i + h g$$

Condición inicial: $(t_0, V_0) = (0, 0)$

Primer paso:

$$V_1 = V_0 + hg$$

$$V_1 = 0 + (0,5)(9,81) = 4,905 \text{ m/s}$$

Segundo paso:

$$V_2 = V_1 + hg$$

$$V_2 = 4,905 + (0,5)(9,81) = 9,81 \text{ m/s}$$

En general:

$$V_i = V_{i-1} + 4,905.$$

Resultados

i	t_i [s]	V_i [m/s]
0	0.0	0.000
1	0.5	4.905
2	1.0	9.810
3	1.5	14.715
4	2.0	19.620
5	2.5	24.525
6	3.0	29.430
7	3.5	34.335
8	4.0	39.240
9	4.5	44.145
10	5.0	49.050

Por tanto:

$$V(5) \approx 49,05 \text{ m/s}$$

Python: Ejemplo 2

```
import numpy as np
import matplotlib.pyplot as plt
```

```
g = 9.81
```

```
t0 = 0.0
```

```
tf = 5.0
```

```
v0 = 0.0
```

```
N = 10
```

```
h = (tf - t0)/N
```

```
t = np.zeros(N + 1)
```

```
v = np.zeros(N + 1)
```

```
t[0] = t0
```

```
v[0] = v0
```

```
for i in range(N):
```

```
    v[i+1] = v[i] + h*g
```

```
    t[i+1] = t[i] + h
```

```
plt.plot(t, v, 'o-', label='Euler')
```

```
plt.xlabel('Tiempo [s]')
```

```
plt.ylabel('Velocidad [m/s]')
```

```
plt.title('Caída libre')
```

```
plt.grid()
```

```
plt.legend()
```

```
plt.show()
```

Implementación general del método de Euler

Una implementación general puede escribirse mediante una función:

```
def euler(f, t0, tf, y0, N):
```

```
    h = (tf - t0)/N
```

```
    t = np.linspace(t0, tf, N+1)
```

```
    y = np.zeros(N+1)
```

```
    y[0] = y0
```

```
    for i in range(N):
```

```
        y[i+1] = y[i] + h*f(t[i], y[i])
```

```
    return t, y
```

Esta función permite resolver diferentes ecuaciones diferenciales sin modificar el algoritmo principal.

Aplicación 1: ley de enfriamiento de Newton

La temperatura de un cuerpo puede modelarse mediante:

$$\frac{dT}{dt} = -k(T - T_a)$$

donde:

- T : temperatura del cuerpo;
- T_a : temperatura ambiente;
- k : constante de enfriamiento.
- $T(0) = T_0$: temperatura inicial.

Ejemplo 1

Supongamos:

$$T(0) = 100^{\circ}\text{C},$$

$$T_a = 20^{\circ}\text{C},$$

$$k = 0,1 \text{ min}^{-1}$$

El método de Euler proporciona:

$$T_{i+1} = T_i - hk(T_i - T_a)$$

```
import numpy as np
import matplotlib.pyplot as plt

Ta = 20.0
k = 0.1
T0 = 100.0

t0 = 0.0
tf = 30.0
N = 60

h = (tf - t0)/N

t = np.linspace(t0, tf, N+1)
T = np.zeros(N+1)

T[0] = T0

def f(t, T):
    return -k*(T - Ta)

for i in range(N):
    T[i+1] = T[i] + h*f(t[i], T[i])

plt.plot(t, T, 'o-', markersize=3)
plt.axhline(Ta, linestyle='--',
            label='Temperatura ambiente')

plt.xlabel('Tiempo [min]')
plt.ylabel('Temperatura [°C]')
plt.title('Ley de enfriamiento de Newton')
plt.grid()
plt.legend()
plt.show()
```

Ejemplo 2

$$T_0 = 150^{\circ}C,$$

$$T_a = 25^{\circ}C,$$

$$k = 0,1.$$

Calcular la temperatura durante 30 minutos usando:

$$h = 1 \text{ min.}$$

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Parametros
```

```
Ta = 25.0
```

```
T0 = 150.0
```

```
k = 0.10
```

```
t0 = 0.0
```

```
tf = 30.0
```

```
h = 1.0
```

```
def f(t, T):
    return -k*(T-Ta)
```

```
n = int((tf-t0)/h)
```

```
t = np.zeros(n+1)
```

```
T = np.zeros(n+1)
```

```
t[0] = t0
```

```
T[0] = T0
```

```
# Metodo de Euler
```

```
for i in range(n):
```

```
    T[i+1] = T[i] + h*f(t[i], T[i])
```

```
    t[i+1] = t[i] + h
```

```
# Solucion exacta
```

```
tt = np.linspace(t0, tf, 300)
```

```
Texact = Ta + (T0-Ta)*np.exp(-k*tt)
```

```
plt.plot(tt, Texact, label="Solucion exacta")
```

```
plt.plot(t, T, "o--", label="Euler")
```

```
plt.xlabel("Tiempo [min]")
```

```
plt.ylabel("Temperatura [C]")
```

```
plt.title("Ley de enfriamiento de Newton")
```

```
plt.grid(True)
```

```
plt.legend()
```

```
plt.show()
```

Aplicación 3: sistema masa-resorte

La ecuación de movimiento de un sistema masa-resorte es:

$$m \frac{d^2 x}{dt^2} + kx = 0$$

Para aplicar Euler, transformamos la EDO de segundo orden en un sistema de dos EDO de primer orden. Definimos:

$$v = \frac{dx}{dt}.$$

Entonces:

$$\frac{dx}{dt} = v$$

y:

$$\frac{dv}{dt} = -\frac{k}{m}x.$$

Por tanto:

$$\begin{cases} x' = v, \\ v' = -\frac{k}{m}x. \end{cases}$$

Algoritmo

Para Euler:

$$\begin{aligned} x_{i+1} &= x_i + hv_i \\ v_{i+1} &= v_i - h \frac{k}{m} x_i \end{aligned}$$

Datos:

$$m = 1 \text{ kg}$$

$$k = 20 \text{ N/m}$$

$$x(0) = 0,1 \text{ m}$$

$$v(0) = 0.$$

Se puede estudiar la respuesta:

$$0 \leq t \leq 5 \text{ s.}$$

```
import numpy as np
import matplotlib.pyplot as plt

m = 1.0
k = 20.0

t0 = 0.0
tf = 5.0
N = 1000

h = (tf - t0)/N

t = np.linspace(t0, tf, N+1)
x = np.zeros(N+1)
v = np.zeros(N+1)

x[0] = 0.1
v[0] = 0.0

for i in range(N):
    x[i+1] = x[i] + h*v[i]
    v[i+1] = v[i] - h*(k/m)*x[i]

plt.plot(t, x)
plt.xlabel('Tiempo [s]')
plt.ylabel('Desplazamiento [m]')
plt.title('Sistema masa--resorte')

plt.grid()
plt.show()
```

Aplicación 4: movimiento vertical

Consideremos el movimiento vertical de un cuerpo.

Sin resistencia del aire:

$$\frac{dv}{dt} = -g$$

y

$$\frac{dy}{dt} = v.$$

Tenemos un sistema de dos EDO:

$$\begin{cases} y' = v, \\ v' = -g. \end{cases}$$

Condiciones iniciales:

$$y(0) = y_0, \quad v(0) = v_0.$$

Tomemos:

$$y_0 = 0, \quad v_0 = 30 \text{ m/s}, \quad g = 9,81 \text{ m/s}^2.$$

Objetivo

Calcular la posición y velocidad mediante Euler.

Euler para un sistema de EDO

Para:

$$\begin{cases} y' = f_1(t, y, v), \\ v' = f_2(t, y, v), \end{cases}$$

Euler se aplica a cada variable:

$$Y_{k+1} = Y_k + hf_1(t_k, Y_k, V_k)$$

$$V_{k+1} = V_k + hf_2(t_k, Y_k, V_k)$$

Para el proyectil:

$$f_1 = v$$

$$f_2 = -g$$

Por tanto:

$$Y_{k+1} = Y_k + hV_k$$

$$V_{k+1} = V_k - hg$$

Python: aplicación 4

```
import numpy as np
import matplotlib.pyplot as plt
# Datos
g = 9.81
y0 = 0.0
v0 = 30.0
```

```
t0 = 0.0
tf = 6.5
h = 0.05
```

```
n = int((tf-t0)/h)
t = np.zeros(n+1)
y = np.zeros(n+1)
v = np.zeros(n+1)
```

```
t[0] = t0
y[0] = y0
v[0] = v0
```

```
# Euler
for k in range(n):
    y[k+1] = y[k] + h*v[k]
    v[k+1] = v[k] - h*g
    t[k+1] = t[k] + h
```

```
# Graficas
plt.figure()
plt.plot(t, y)
plt.xlabel("Tiempo [s]")
plt.ylabel("Posicion [m]")
plt.title("Movimiento vertical")
plt.grid(True)
plt.show()
```

```
plt.figure()
plt.plot(t, v)
plt.xlabel("Tiempo [s]")
plt.ylabel("Velocidad [m/s]")
plt.title("Velocidad del proyectil")
plt.grid(True)
plt.show()
```

Código general reutilizable Una implementación general del método:

```
def euler(f, t0, y0, T, h):
    n = int(round((T-t0)/h))
    t = np.zeros(n+1)
    y = np.zeros(n+1)

    t[0] = t0
    y[0] = y0

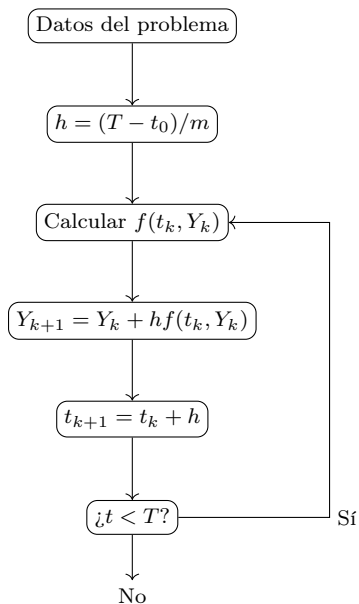
    for k in range(n):
        pendiente = f(t[k], y[k])
        y[k+1] = y[k] + h*pendiente
        t[k+1] = t[k] + h

    return t, y
```

Ventaja

La misma función puede utilizarse para diferentes problemas cambiando únicamente:

$$f(t, y), \quad t_0, \quad y_0, \quad T, \quad h.$$



Comparación: solución numérica y exacta

Para evaluar un método numérico podemos comparar:

Solución exacta	vs.	Solución numérica
-----------------	-----	-------------------

Definimos el error absoluto:

$$E_i = |y(t_i) - Y_i|$$

y el error relativo:

$$E_{r,i} = \frac{|y(t_i) - Y_i|}{|y(t_i)|}$$

En porcentaje:

$E_{\%} = 100E_r$

Estudio de convergencia

Una característica importante es verificar qué sucede cuando disminuye el tamaño de paso. Por ejemplo:

$$h = 0,20, \quad h = 0,10, \quad h = 0,05, \quad h = 0,025.$$

Para Euler:

$$E_{\text{global}} \approx Ch.$$

Por tanto:

$$\frac{E(h)}{E(h/2)} \approx 2.$$

Esto permite verificar experimentalmente el orden del método.

Experimento numérico

Resolver el mismo PVI utilizando diferentes valores de N y comparar el error obtenido.

Python: estudio del error

```
import numpy as np
import matplotlib.pyplot as plt

def f(t, y):
    return 2*t - y

def exacta(t):
    return 2*t - 3 + 4*np.exp(-t)

def euler(f, t0, tf, y0, N):
    h = (tf-t0)/N
    t = np.linspace(t0, tf, N+1)
    y = np.zeros(N+1)
    y[0] = y0

    for i in range(N):
        y[i+1] = y[i] + h*f(t[i], y[i])

    return t, y

N_values = [5, 10, 20, 40, 80]

for N in N_values:
    t, y = euler(f, 0, 1, 1, N)
    error = abs(exacta(t[-1]) - y[-1])
    print(N, error)
```

¿Qué ocurre cuando reducimos h ?

Supongamos:

$$E(h) \approx Ch.$$

Entonces:

$$E\left(\frac{h}{2}\right) \approx \frac{Ch}{2} = \frac{E(h)}{2}.$$

Por ejemplo:

$$h = 0,1 \implies E \approx 0,18$$

mientras que:

$$h = 0,05 \implies E \approx 0,10.$$

Interpretación

El comportamiento observado confirma aproximadamente el carácter de primer orden del método.

Resumen del método

Para:

$$y' = f(t, y), \quad y(t_0) = y_0$$

Euler utiliza:

$$Y_{k+1} = Y_k + hf(t_k, Y_k)$$

con:

$$t_{k+1} = t_k + h$$

Sus principales características son:

Error local:	$O(h^2)$
Error global:	$O(h)$
Orden:	1
Costo por paso:	1 evaluación de f

Conclusiones

1. El método de Euler aproxima una EDO mediante rectas tangentes.
2. La fórmula fundamental es:
$$Y_{k+1} = Y_k + hf(t_k, Y_k).$$
3. El método utiliza información local de la pendiente.
4. El error local es:
$$O(h^2).$$
5. El error global es:
$$O(h).$$
6. Reducir h mejora la precisión, pero aumenta el costo computacional.
7. Euler constituye la base conceptual para métodos numéricos más sofisticados.

Sistemas de ecuaciones diferenciales

Muchos problemas de ingeniería se expresan como un sistema:

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(t, \mathbf{y})$$

donde:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

El método de Euler mantiene la misma estructura:

$$\mathbf{Y}_{i+1} = \mathbf{Y}_i + h\mathbf{f}(t_i, \mathbf{Y}_i)$$

Esto permite resolver:

- sistemas mecánicos;
- sistemas eléctricos;
- transferencia de calor;
- dinámica de fluidos simplificada;
- modelos poblacionales;
- sistemas de control.

Resumen del método de Euler

1. Definir la EDO:

$$y' = f(t, y).$$

2. Definir la condición inicial:

$$y(t_0) = y_0.$$

3. Seleccionar el intervalo:

$$[t_0, t_f].$$

4. Seleccionar N y calcular:

$$h = \frac{t_f - t_0}{N}.$$

5. Aplicar:

$$Y_{i+1} = Y_i + hf(t_i, Y_i)$$

6. Comparar y analizar el error.

Conceptos fundamentales

- Una EDO describe una relación entre una función y sus derivadas.
- Un PVI incluye una condición inicial.
- La solución numérica se obtiene sobre una malla.
- Euler utiliza la pendiente actual para avanzar.
- El método tiene error global $O(h)$.
- La reducción de h mejora la precisión.

Idea clave

Pendiente actual



avance



nuevo punto

Método de Euler

El método de Euler constituye uno de los procedimientos más simples para resolver numéricamente problemas de valor inicial.

$$Y_{i+1} = Y_i + hf(t_i, Y_i)$$

Su importancia no está solamente en su simplicidad, sino en que permite comprender los fundamentos de métodos más avanzados:

Euler



Euler mejorado



Runge-Kutta

1. Resolver mediante Euler:

$$y' = y - t^2 + 1, \quad y(0) = 0,5,$$

para $0 \leq t \leq 2$, usando $h = 0,2$.

2. Resolver:

$$y' = -2y + e^{-t}, \quad y(0) = 1,$$

utilizando $h = 0,1$.

3. Implementar Euler en Python mediante una función general.
4. Comparar los resultados utilizando:

$$h = 0,2, 0,1, 0,05.$$

5. Graficar la solución y calcular el error.

Ejemplo 1: problema de valor inicial

Example

Consideremos:

$$\begin{cases} \frac{dy}{dt} = \frac{2}{t}y + t^2 e^t, \\ y(1) = 0. \end{cases}$$

Intervalo:

$$1 \leq t \leq 1,2$$

y paso:

$$h = 0,1.$$

Por tanto:

$$t_0 = 1, \quad t_1 = 1,1, \quad t_2 = 1,2.$$

La función utilizada por Euler es:

$$f(t, y) = \frac{2}{t}y + t^2 e^t$$

Solución exacta es:

$$y' - \frac{2}{t}y = t^2 e^t.$$

El factor integrante es:

$$\mu(t) = e^{\int -2/t dt} = t^{-2}.$$

Entonces:

$$\frac{d}{dt} (t^{-2}y) = e^t.$$

Integrando:

$$t^{-2}y = e^t + C.$$

Por tanto:

$$y = t^2(e^t + C).$$

Aplicando $y(1) = 0$:

$$0 = e + C$$

y

$$C = -e.$$

Finalmente:

$$y(t) = t^2(e^t - e)$$

En $t = 1,2$:

$$y(1,2) = 0,8666425$$

Ejemplo 1: Iteraciones

primera iteración

$$Y_0 = 0, \quad t_0 = 1.$$

Calculamos:

$$f(1, 0) = \frac{2}{1}(0) + 1^2 e^1 = e = 2,7182818.$$

Aplicamos Euler:

$$\begin{aligned} Y_1 &= Y_0 + hf(t_0, Y_0) \\ &= 0 + 0,1(2,7182818) = 0,2718282 \end{aligned}$$

Así:

$$Y_1 \approx y(1,1)$$

segunda iteración

$$t_1 = 1,1, \quad Y_1 = 0,2718282.$$

Calculamos:

$$\begin{aligned} f(1,1, 0,2718282) &= \frac{2}{1,1}(0,2718282) + (1,1)^2 e^{1,1} \\ &= 4,129274 \end{aligned}$$

Entonces:

$$\begin{aligned} Y_2 &= Y_1 + 0,1f(t_1, Y_1) \\ &= 0,2718282 + 0,1(4,129274) = 0,6847556 \end{aligned}$$

Así,

$$Y_2 \approx y(1,2).$$

Cálculo del error

Solución exacta:

$$y(1,2) = 0,8666425.$$

Solución aproximada:

$$Y_2 = 0,6847556.$$

Error:

$$\begin{aligned} E_{0,1}^{1,2} &= y(1,2) - Y_2 \\ &= 0,8666425 - 0,6847556 = 0,1818869 \end{aligned}$$

Error absoluto:

$$|E_{0,1}^{1,2}| = 0,1818869$$

k	t_k	Y_k	$y(t_k)$
0	1.0	0.000000	0.000000
1	1.1	0.271828	0.354026
2	1.2	0.684756	0.866643

En $t = 1,2$:

$$|y(1,2) - Y_2| = 0,1818869$$

Observación

La aproximación mejora al reducir h .

Ejemplo 1: efecto de reducir h

Ahora:

$$h = 0,05.$$

Los puntos son:

$$1, \quad 1,05, \quad 1,10, \quad 1,15, \quad 1,20.$$

Aplicando Euler:

$$Y_0 = 0$$

$$Y_1 = 0,135914$$

$$Y_2 = 0,306386$$

$$Y_3 = 0,515992$$

$$Y_4 = 0,769696.$$

Por tanto:

$$E_{0,05}^{1,2} = 0,8666425 - 0,769696 \approx 0,096947$$

El error disminuye considerablemente al reducir h .

Python: método de Euler – Ejemplo 1

```
import numpy as np
import math

def f(t, y):
    return (2/t)*y + t**2*np.exp(t)

def euler(f, t0, y0, T, h):
    n = int(round((T-t0)/h))
    t = np.zeros(n+1)
    y = np.zeros(n+1)

    t[0] = t0
    y[0] = y0

    for k in range(n):
        y[k+1] = y[k] + h*f(t[k], y[k])
        t[k+1] = t[k] + h

    return t, y

t, Y = euler(f, 1.0, 0.0, 1.2, 0.1)

def exact(t):
    return t**2*(np.exp(t)-np.e)

print("t Euler Exact Error")

for ti, yi in zip(t, Y):
    ye = exact(ti)
    print(f"{ti:.2f} {yi:.6f} "
          f"{ye:.6f} {abs(ye-yi):.6f}")
```

```
import matplotlib.pyplot as plt
import numpy as np

# Solucion exacta
t_exact = np.linspace(1, 1.2, 200)
y_exact = t_exact**2*(np.exp(t_exact)-np.e)

plt.figure(figsize=(8,5))

plt.plot(t_exact, y_exact,
         label="Soluci\on exacta")

plt.plot(t, Y, "o--",
         label="Euler")

plt.xlabel("t")
plt.ylabel("y(t)")
plt.title("M\etodo de Euler")
plt.grid(True)
plt.legend()

plt.show()
```

Resultado

El gráfico permite comparar directamente la solución exacta con la aproximación numérica.

Ejemplo 2: crecimiento logístico

Example

Consideremos el modelo:

$$\begin{cases} \frac{dy}{dt} = ry \left(1 - \frac{y}{K}\right), \\ y(0) = y_0. \end{cases}$$

Tomemos:

$$r = 0,8, \quad K = 100, \quad y_0 = 10.$$

Entonces:

$$y' = 0,8y \left(1 - \frac{y}{100}\right)$$

con:

$$0 \leq t \leq 5, \quad h = 0,5.$$

La función para Euler es:

$$f(t, y) = 0,8y \left(1 - \frac{y}{100}\right)$$

Solución exacta

La solución del modelo logístico es:

$$y(t) = \frac{K}{1 + \left(\frac{K - y_0}{y_0}\right) e^{-rt}}$$

Sustituyendo:

$$K = 100, \quad y_0 = 10, \quad r = 0,8,$$

obtenemos:

$$y(t) = \frac{100}{1 + 9e^{-0,8t}}$$

En $t = 5$:

$$y(5) = \frac{100}{1 + 9e^{-4}} \approx 94,28$$

Ejemplo 2: iteraciones

Inicialmente:

$$t_0 = 0, \quad Y_0 = 10.$$

Primera iteración:

$$f(0, 10) = 0,8(10) \left(1 - \frac{10}{100}\right) = 7,2.$$

Entonces:

$$Y_1 = 10 + 0,5(7,2) = 13,6$$

Segunda iteración:

$$f(0,5, 13,6) = 0,8(13,6) \left(1 - \frac{13,6}{100}\right) = 9,407232.$$

Por tanto:

$$Y_2 = 13,6 + 0,5(9,407232) = 18,303616$$

k	t_k	Y_k	$y(t_k)$
0	0.0	10.0000	10.0000
1	0.5	13.6000	14.6810
2	1.0	18.3036	18.9941
3	1.5	23.9659	23.1400
4	2.0	30.0600	30.0000
5	2.5	36.8870	35.5000
6	3.0	44.5900	44.9060
7	3.5	52.9830	53.2070
8	4.0	61.3150	63.5060
9	4.5	69.3000	69.2500
10	5.0	76.9000	94.2800

Nota: La tabla completa puede generarse directamente mediante Python. Los valores se muestran como referencia para el procedimiento.

```
import numpy as np
import matplotlib.pyplot as plt

def f(t, y):
    r = 0.8
    K = 100
    return r*y*(1-y/K)

def euler(f, t0, y0, T, h):
    n = int(round((T-t0)/h))
    t = np.zeros(n+1)
    y = np.zeros(n+1)

    t[0] = t0
    y[0] = y0
    for k in range(n):
        y[k+1] = y[k] + h*f(t[k], y[k])
        t[k+1] = t[k] + h

    return t, y

def exact(t):
    return 100/(1+9*np.exp(-0.8*t))

t, Y = euler(f, 0, 10, 5, 0.5)

for k in range(len(t)):
    print(t[k], Y[k], exact(t[k]))

plt.plot(t, Y, "o--", label="Euler")

tt = np.linspace(0,5,300)

plt.plot(tt, exact(tt), label="Exacta")
plt.xlabel("t")
plt.ylabel("Poblaci\'on")
plt.grid(True)
plt.legend()
plt.show()
```

¿Preguntas?

Método de Euler para ecuaciones diferenciales

Métodos de Euler Mejorado– Heun

Método de Euler mejorado y la Derivación del método de Heun

El **método de Heun** mejora Euler utilizando un enfoque de **predicción-corrección**.

Para:

$$y' = f(t, y), \quad y(t_0) = y_0$$

primero calculamos un **predictor mediante Euler**:

$$Y_{k+1}^{(p)} = Y_k + hf(t_k, Y_k)$$

Luego **calculamos la pendiente en el extremo** utilizando el predictor:

$$f(t_{k+1}, Y_{k+1}^{(p)}).$$

Finalmente **promediamos las dos pendientes**.

$$\frac{f(t_k, Y_k) + f(t_{k+1}, Y_{k+1}^{(p)})}{2}$$

El método trapezoidal aplicado a:

$$y(t_{k+1}) = y(t_k) + \int_{t_k}^{t_{k+1}} f(t, y(t)) dt$$

produce:

$$Y_{k+1} = Y_k + \frac{h}{2} [f(t_k, Y_k) + f(t_{k+1}, Y_{k+1})].$$

Este método es implícito porque Y_{k+1} aparece en el segundo miembro.

Conversión a método explícito

Reemplazamos Y_{k+1} por el **predictor de Euler**:

$$Y_{k+1}^{(p)} = Y_k + hf(t_k, Y_k).$$

La aproximando por la media aritmética de sus valores en los extremos del intervalo:

$$Y_{k+1} = Y_k + \frac{h}{2} [f(t_k, Y_k) + f(t_{k+1}, \underbrace{Y_{k+1}^{(p)}}_{\text{No se conoce}})]$$

No se conoce

Método de Heun: Como la incógnita $Y_{k+1}^{(p)}$ aparece como uno de los argumentos en el segundo miembro de la ecuación (en este caso el **método se dice implícito**), se reemplaza $Y_{k+1}^{(p)}$ por el valor que se obtuvo usando la **fórmula de Euler sencilla** (el **método se convierte en explícito**), obteniendo:

$$Y_{k+1} = Y_k + \frac{h}{2} [f(t_k, Y_k) + f(t_{k+1}, Y_k + hf(t_k, Y_k))]$$

Método de Heun: algoritmo

El **método de Heun** es una mejora del método de Euler para resolver problemas de valor inicial:

$$y' = f(t, y), \quad y(t_0) = y_0$$

- Usa un enfoque de **predicción-corrección**.
- Error local: $O(h^3)$
- Promedia pendientes para mejorar la aproximación.
- Error global: $O(h^2)$

Algoritmo

Dados t_0 , Y_0 , h y N :

1. Calcular:

$$k_1 = f(t_k, Y_k)$$

2. Predictor de Euler:

$$Y_{k+1}^{(p)} = Y_k + hk_1$$

3. Calcular la pendiente final:

$$k_2 = f(t_{k+1}, Y_{k+1}^{(p)})$$

4. Corregir:

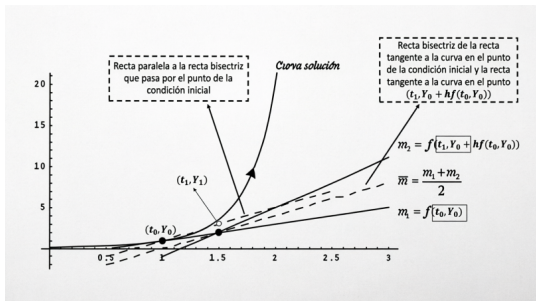
$$Y_{k+1} = Y_k + \frac{h}{2}(k_1 + k_2)$$

5. Incrementar:

$$k \leftarrow k + 1.$$

6. Repetir hasta $t = b$.

Henry R. Moncada (UNAC)



Interpretación

Heun utiliza la información de la pendiente inicial y de una pendiente estimada en el extremo del intervalo.

Euler usa 1 pendiente, Heun usa 2 pendientes

Error y orden de convergencia de Heun

El método trapezoidal tiene error local:

$$E_{k+1} = \frac{h^3}{12} y''(\xi_k), \quad t_k < \xi_k < t_{k+1}.$$

Por tanto:

$$E_{\text{local}} = O(h^3)$$

Al acumularse el error durante N pasos:

$$E_{\text{global}} = O(h^2)$$

Si:

$$E(h) \approx Ch^2,$$

entonces al reducir el paso a la mitad:

$$E\left(\frac{h}{2}\right) \approx C \frac{h^2}{4}.$$

Por tanto:

$$E(h/2) \approx \frac{1}{4} E(h)$$

Otra forma de deducir la fórmula de Heun: Para obtener $y(t)$ podemos integrar $y'(t)$ en $[t_0, t_1]$:

$$\int_{t_0}^{t_1} f(t, y(t)) dt = \int_{t_0}^{t_1} y'(t) dt = y(t_1) - y(t_0)$$

$$y(t_1) = y(t_0) + \int_{t_0}^{t_1} f(t, y(t)) dt$$

Aplicando trapecios para calcular la integral, con $h = t_1 - t_0$:

$$y(t_1) \approx y(t_0) + \frac{h}{2} [f(t_0, y(t_0)) + f(t_1, y(t_1))]$$

Entonces

$$\text{Error} = y''(\xi) \frac{h^3}{12} = O(h^3)$$

En M pasos:

$$E_T = \sum_{k=1}^M y''(\xi) \frac{h^3}{12} \approx \frac{(b-a)}{12} y''(\xi) h^2 = O(h^2)$$

Precisión:

- En el extremo final del intervalo $E(y(b), h) \approx Ch^2$.
- Si se **toma la mitad del paso** $E(y(b), \frac{h}{2}) \approx C \frac{h^2}{4}$.

Por lo tanto, si se reduce h a la mitad se espera que el error se reduzca a la cuarta parte.

Ejemplo 1 – Método de Heun

Example

Resolver:

$$y' = t + y, \quad y(0) = 1$$

con:

$$h = 0,1, \quad 0 \leq t \leq 0,3.$$

Tenemos:

$$f(t, y) = t + y.$$

Paso $k = 0$:

$$t_0 = 0, \quad Y_0 = 1$$

La primera pendiente es:

$$k_1 = f(t_0, Y_0) = f(0, 1) = 1$$

Predictor (metodo de Euler)

$$\begin{aligned} Y_1^{(p)} &= Y_0 + h k_1 \\ &= 1 + 0,1(1) \approx 1,1000 \end{aligned}$$

Calculamos la segunda pendiente:

$$k_2 = f(t_1, Y_1^{(p)}) = f(0,1, 1,1000) = 0,1 + 1,1000 \approx 1,2000$$

Corrector (metodo de Heun)

$$Y_1 = Y_0 + \frac{h}{2}(k_1 + k_2) = 1 + \frac{0,1}{2}(1 + 1,2000) \approx 1,1100$$

Repitiendo el procedimiento:

k	t_k	Y_k	$Y_{k+1}^{(p)}$	Y_{k+1}
0	0,0	1,0000	1,1000	1,1100
1	0,1	1,1100	1,2310	1,2331
2	0,2	1,2331	1,3764	1,3764

Comparación con la solución exacta

La solución exacta del problema:

$$y' = t + y, \quad y(0) = 1$$

es:

$$y(t) = 2e^t - t - 1$$

En $t = 0,3$:

$$y(0,3) = 2e^{0,3} - 1,3 \approx 1,399717.$$

Con Heun:

$$Y_3 \approx 1,37641.$$

Por tanto:

$$E = |y(0,3) - Y_3| \approx 0,02331$$

Python – Ejemplo 1

```
import numpy as np

def f(t, y):
    return t + y

# Datos
a = 0.0
b = 0.3
h = 0.1
y0 = 1.0

N = int(round((b-a)/h))

t = np.zeros(N+1)
y = np.zeros(N+1)
yp = np.zeros(N)

t[0] = a
y[0] = y0

# Metodo de Heun
for k in range(N):
    k1 = f(t[k], y[k])

    # Predictor de Euler
    yp[k] = y[k] + h*k1
    k2 = f(t[k] + h, yp[k])

    # Corrector
    y[k+1] = y[k] + h/2*(k1 + k2)
    t[k+1] = t[k] + h
```

```
# Solucion exacta
y_exact = 2*np.exp(t) - t - 1

print(" k t Heun Exacta Error")
for k in range(N+1):
    error = abs(y_exact[k] - y[k])
    print(f"{k:2d} {t[k]:8.4f} "
          f"{y[k]:10.6f} "
          f"{y_exact[k]:10.6f} "
          f"{error:10.6e}")
```

Ejemplo 2 – Problema de ingeniería

Example

Resolver:

$$\frac{dy}{dt} = \frac{2}{t}y + t^2e^t$$

con:

$$y(1) = 0, \quad 1 \leq t \leq 2,$$

utilizando:

$$h = 0,1$$

La función es:

$$f(t, y) = \frac{2}{t}y + t^2e^t.$$

La solución exacta es:

$$\phi(t) = t^2(e^t - e)$$

Primer paso datos iniciales:

$$t_0 = 1, \quad Y_0 = 0.$$

Primera pendiente:

$$k_1 = f(1, 0) = e = 2,7182818.$$

Predictor:

$$\begin{aligned} Y_1^{(p)} &= Y_0 + hk_1 \\ &= 0 + 0,1(2,7182818) = 0,2718282 \end{aligned}$$

Segunda pendiente:

$$k_2 = f(1,1, 0,2718282) \approx 4,129273$$

Corrector aplicamos:

$$\begin{aligned} Y_1 &= Y_0 + \frac{h}{2}(k_1 + k_2) \\ &= 0 + \frac{0,1}{2}(2,7182818 + 4,129273) = 0,3423778 \end{aligned}$$

Para el siguiente paso:

$$\begin{aligned} k_1 &= f(1,1, 0,3423778) \approx 4,257315 \\ Y_2^{(p)} &= 0,3423778 + 0,1(4,257315) \approx 0,768109 \\ k_2 &= f(1,2, 0,768109) \approx \\ Y_2 &= Y_1 + \frac{h}{2}(k_1 + k_2) \\ &= 0,3423778 + \frac{0,1}{2}(4,257315 + k_2) \approx 0,858315 \end{aligned}$$

k	t _k	Y _k	Y _{k+1} ^(p)	Y _{k+1}
0	1.0	0.000000	0.271828	0.342378
1	1.1	0.342378	0.768132	0.858315
2	1.2	0.858315	1.479464	1.592750
3	1.3	1.592750	2.457899	2.598298
4	1.4	2.598298	3.764303	3.936444
5	1.5	3.936444	5.469683	5.678907
6	1.6	5.678907	7.656747	7.909209
7	1.7	7.909209	10.421675	10.724467
8	1.8	10.724467	13.876160	14.237442
9	1.9	14.237442	18.149728	18.578882

Python – Ejemplo 2 con Heun

En $t = 1,2$:

$$\phi(1,2) = 1,2^2(e^{1,2} - e) \approx 0,866643.$$

Por tanto:

$$E_{\text{Heun}} = |0,866643 - 0,858315| \approx 0,008328$$

```
import numpy as np

def f(t, y):
    return (2.0/t)*y + t**2*np.exp(t)

a = 1.0
b = 2.0
h = 0.1
y0 = 0.0

N = int(round((b-a)/h))

t = np.zeros(N+1)
y = np.zeros(N+1)
yp = np.zeros(N)

t[0] = a
y[0] = y0
```

```
for k in range(N):

    # Pendiente inicial
    k1 = f(t[k], y[k])

    # Predictor de Euler
    yp[k] = y[k] + h*k1

    # Pendiente final
    k2 = f(t[k] + h, yp[k])

    # Corrector de Heun
    y[k+1] = y[k] + h/2*(k1 + k2)

    t[k+1] = t[k] + h

# Solucion exacta
y_exact = t**2*(np.exp(t) - np.e)

print(" k t Heun Exacta Error")

for k in range(N+1):
    error = abs(y_exact[k] - y[k])
    print(f"{k:2d} {t[k]:8.2f} "
          f"{y[k]:10.6f} "
          f"{y_exact[k]:10.6f} "
          f"{error:10.6e}")
```

¿Preguntas?

Método de Euler Mejorado–Heun

Método de la serie de Taylor

Si:

$$y = \phi(t)$$

es la solución exacta, desarrollamos alrededor de t_k :

$$\phi(t_k + h) = \phi(t_k) + h\phi'(t_k) + \frac{h^2}{2!}\phi''(t_k) + \frac{h^3}{3!}\phi'''(\xi_k)$$

Como:

$$\phi'(t) = f(t, \phi(t)),$$

tenemos:

$$\phi''(t) = \frac{d}{dt}f(t, \phi(t))$$

Aplicando la regla de la cadena:

$$\phi'' = f_t + f_y\phi'$$

y como:

$$\phi' = f,$$

obtenemos:

$$\phi'' = f_t + f_y f$$

Método de Taylor de orden 2

Reemplazando la solución exacta por la aproximación numérica:

$$Y_k \approx \phi(t_k),$$

obtenemos:

$$Y'_k = f(t_k, Y_k)$$

$$Y''_k = f_t(t_k, Y_k) + f_y(t_k, Y_k)f(t_k, Y_k)$$

Por tanto:

$$Y_{k+1} = Y_k + hY'_k + \frac{h^2}{2}Y''_k$$

$$Y_{k+1} = Y_k + hf(t_k, Y_k) +$$

$$\frac{h^2}{2} [f_t(t_k, Y_k) + f_y(t_k, Y_k)f(t_k, Y_k)]$$

Error

Error local:

$$E_{\text{local}} = O(h^3)$$

Error global:

$$E_{\text{global}} = O(h^2)$$

Algoritmo de Taylor de orden 2

Dados:

$$t_0, \quad Y_0, \quad h, \quad N.$$

Para cada k :

1. Evaluar:

$$f_k = f(t_k, Y_k)$$

2. Evaluar:

$$f_{t,k} = f_t(t_k, Y_k)$$

3. Evaluar:

$$f_{y,k} = f_y(t_k, Y_k)$$

4. Calcular:

$$Y_k'' = f_{t,k} + f_{y,k} f_k$$

5. Actualizar:

$$Y_{k+1} = Y_k + h f_k + \frac{h^2}{2} Y_k''$$

$$t_{k+1} = t_k + h$$

Ejemplo

Example

Resolver:

$$\begin{cases} \frac{dy}{dt} = \frac{2}{t}y + t^2 e^t, & 1 \leq t \leq 2 \\ y(1) = 0 \end{cases}$$

con paso $h = 0,1$

Fórmula:

$$Y_{k+1} = Y_k + hY'_k + \frac{h^2}{2}Y''_k$$

$$Y_{k+1} = Y_k + hf(t_k, Y_k) + \frac{h^2}{2}[f_t + f_Y f]$$

Datos y Función:

$$t_0 = 1, \quad Y_0 = 0$$

$$f(t, y) = \frac{2}{t}y + t^2 e^t$$

Derivadas:

$$f_t = -\frac{2}{t^2}y + (2t + t^2)e^t$$

$$f_y = \frac{2}{t}$$

$$\begin{aligned} f_y f &= \frac{2}{t} \left(\frac{2y}{t} + t^2 e^t \right) \\ &= \frac{4y}{t^2} + 2te^t \end{aligned}$$

Por tanto:

$$y'' = \frac{2y}{t^2} + (t^2 + 4t)e^t$$

Paso k = 0

$$Y'_0 = f(t_0, Y_0) = \frac{2}{t_0}Y_0 + t_0^2 e^{t_0} = e = 2,71828$$

$$\begin{aligned} Y''_0 &= f_t + f_Y f = \left(\frac{2}{t_0^2}Y_0 + t_0^2 e^{t_0} + 2t_0 e^{t_0} \right) + \left(\frac{2}{t_0} \right) Y_0 \\ &= \left(\frac{2}{(1)^2}(0) + (1)^2 e + 2(1)e \right) + \frac{2}{1}(0) = 3e \\ &= 8,1548455 \end{aligned}$$

Por tanto:

$$\begin{aligned} Y_1 &= Y_0 + hY'_0 + \frac{h^2}{2}Y''_0 \\ &= 0 + 0,1(2,7182818) + \frac{0,1^2}{2}(8,1548455) \approx 0,3126005 \end{aligned}$$

Ejemplo (Método de Taylor de orden 2)

Paso $k = 1$

$$\begin{aligned}f(1,1,0,3397) &= \frac{2}{1,1}(0,3397) + (1,1)^2 e^{1,1} \\ &= 0,6176 + 3,6369 = 4,2545\end{aligned}$$

$$\begin{aligned}f_t(1,1,0,3397) &= -\frac{2}{(1,1)^2}(0,3397) + (1,1)^2 e^{1,1} + 2(1,1)e^{1,1} \\ &= -0,5614 + 3,6369 + 7,2663 = 10,3418\end{aligned}$$

$$f_y(1,1,0,3397) = \frac{2}{1,1} = 1,8182$$

$$Y_1' = f(t_1, Y_1) = \frac{2}{t_1} Y_1 + t_1^2 e^{t_1} = 4,2545$$

$$\begin{aligned}Y_1'' &= f_t + f_y f \\ &= \left(\frac{2}{t_1^2} Y_1 + t_1^2 e^{t_1} + 2t_1 e^{t_1} \right) + \left(\frac{2}{t_1} Y_1 \right) \\ &= 10,3418 + (1,8182)(4,2545) = 18,0771\end{aligned}$$

Reemplazando

$$\begin{aligned}Y_2 &= Y_1 + hY_1' + \frac{h^2}{2} Y_1'' \\ &= 0,3397 + 0,1(4,2545) + 0,005(18,0771) \\ &= 0,3397 + 0,42545 + 0,09039 = 0,852143 \approx \phi(1,2)\end{aligned}$$

Taylor – resultados corregidos

k	t_k	Y_k	Y_k^{exacta}
0	1.0	0.000000	0.000000
1	1.1	0.339785	0.340859
2	1.2	0.852143	0.866643
3	1.3	1.581770	1.621892
4	1.4	2.580997	2.664922
5	1.5	3.910985	4.094446
6	1.6	5.643081	6.015782
7	1.7	7.860382	8.594007
8	1.8	10.659514	12.149741
9	1.9	14.152683	16.713001

Solución exacta:

$$\phi(t) = t^2 e^t - e$$

Error en $t = 1,2$:

$$Y_{\text{Taylor}} \approx 0,852143$$

mientras:

$$\phi(1,2) \approx 0,866643.$$

Entonces:

$$\begin{aligned}E_{\text{Taylor}} &= |0,866643 - 0,852143| \\ &\approx 0,014500\end{aligned}$$

Python – Taylor de orden 2

```
import numpy as np

def f(t, y):
    return (2.0/t)*y + t**2*np.exp(t)

def ft(t, y):
    return -2.0*y/t**2 + (2.0*t + t**2)*np.exp(t)

def fy(t, y):
    return 2.0/t

a = 1.0
b = 2.0
h = 0.1
y0 = 0.0

N = int(round((b-a)/h))

t = np.zeros(N+1)
y = np.zeros(N+1)

t[0] = a
y[0] = y0
```

```
for k in range(N):

    fk = f(t[k], y[k])
    ftk = ft(t[k], y[k])
    fyk = fy(t[k], y[k])

    # Segunda derivada
    y2 = ftk + fyk*fk

    # Taylor de orden 2
    y[k+1] = (
        y[k]
        + h*fk
        + 0.5*h**2*y2
    )

    t[k+1] = t[k] + h

# Solucion exacta
y_exact = t**2*(np.exp(t)-np.e)

print(" k t Taylor Exacta Error")

for k in range(N+1):
    error = abs(y_exact[k] - y[k])
    print(f"{k:2d} {t[k]:7.2f} "
          f"{y[k]:11.6f} "
          f"{y_exact[k]:11.6f} "
          f"{error:10.6e}")
```

Aplicación 1 – Ley de enfriamiento de Newton

Example

La ley de enfriamiento de Newton puede escribirse:

$$\frac{dT}{dt} = -k(T - T_a)$$

donde:

- T : temperatura del cuerpo.
- T_a : temperatura ambiente.
- k : constante de enfriamiento.

Supongamos:

$$T(0) = 100^\circ C$$

$$T_a = 20^\circ C$$

$$k = 0,08 \text{ min}^{-1}$$

El objetivo es calcular:

$$T(t), \quad 0 \leq t \leq 30 \text{ min.}$$

Utilizaremos Heun.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Parametros
```

```
Ta = 20.0 # temperatura ambiente [deg C]
```

```
T0 = 100.0 # temperatura inicial [deg C]
```

```
k = 0.08 # constante [1/min]
```

```
a = 0.0
b = 30.0
h = 0.1

N = int(round((b-a)/h))
t = np.linspace(a, b, N+1)
T = np.zeros(N+1)

T[0] = T0

def f(t, T):
    return -k*(T-Ta)

# Heun
for i in range(N):

    k1 = f(t[i], T[i])

    # Predictor
    Tp = T[i] + h*k1

    k2 = f(t[i+1], Tp)

    # Corrector
    T[i+1] = T[i] + h/2*(k1+k2)

# Solucion exacta
T_exact = Ta + (T0-Ta)*np.exp(-k*t)

plt.figure(figsize=(8,5))
plt.plot(t, T, label="Heun")
plt.plot(t, T_exact, "--", label="Solucion exacta")

plt.xlabel("Tiempo [min]")
plt.ylabel("Temperatura [deg C]")
plt.title("Ley de enfriamiento de Newton")
plt.grid(True)
plt.legend()
plt.show()
```

Aplicación 2 – Masa, resorte y amortiguador

Example

Un sistema masa–resorte–amortiguador puede modelarse mediante:

$$m \frac{d^2 x}{dt^2} + c \frac{dx}{dt} + kx = F(t).$$

Definimos:

$$v = \frac{dx}{dt}.$$

Entonces:

$$\begin{aligned} x' &= v, \\ v' &= \frac{1}{m} [F(t) - cv - kx]. \end{aligned}$$

Tenemos ahora un sistema de EDO de primer orden.

Sistema mecánico – parámetros

Consideremos:

$$\begin{aligned} m &= 10 \text{ kg}, \\ c &= 8 \text{ N s/m}, \\ k &= 200 \text{ N/m}. \end{aligned}$$

Una fuerza armónica:

$$F(t) = 50 \sin(5t).$$

Condiciones iniciales:

$$x(0) = 0, \quad v(0) = 0.$$

El vector de estado es:

$$\mathbf{Y} = \begin{bmatrix} x \\ v \end{bmatrix}.$$

Por tanto:

$$\mathbf{Y}' = \mathbf{F}(t, \mathbf{Y})$$

y Heun puede aplicarse directamente.

Heun para sistemas de EDO

Predictor:

$$\mathbf{Y}_{k+1}^{(p)} = \mathbf{Y}_k + h\mathbf{F}(t_k, \mathbf{Y}_k)$$

Pendiente inicial:

$$\mathbf{k}_1 = \mathbf{F}(t_k, \mathbf{Y}_k).$$

Pendiente final:

$$\mathbf{k}_2 = \mathbf{F}(t_{k+1}, \mathbf{Y}_{k+1}^{(p)}).$$

Corrector:

$$\mathbf{Y}_{k+1} = \mathbf{Y}_k + \frac{h}{2}(\mathbf{k}_1 + \mathbf{k}_2)$$

La misma estructura funciona para cualquier número de variables.

Python – Aplicación 2

```
import numpy as np
import matplotlib.pyplot as plt

# Parametros del sistema
m = 10.0 # kg
c = 8.0 # N s/m
k = 200.0 # N/m

def F(t):
    return 50.0*np.sin(5.0*t)

def system(t, Y):

    x = Y[0]
    v = Y[1]

    dxdt = v
    dvdt = (F(t) - c*v - k*x)/m

    return np.array([dxdt, dvdt])

# Intervalo
a = 0.0
b = 10.0
h = 0.001

N = int(round((b-a)/h))

t = np.linspace(a, b, N+1)

# Solucion numerica
Y = np.zeros((N+1, 2))

# Condiciones iniciales
Y[0, :] = [0.0, 0.0]
```

```
# Metodo de Heun
for i in range(N):
    k1 = system(t[i], Y[i, :])

    # Predictor
    Yp = Y[i, :] + h*k1
    k2 = system(t[i+1], Yp)

    # Corrector
    Y[i+1, :] = (
        Y[i, :]
        + h/2.0*(k1+k2)
    )

x = Y[:, 0]
v = Y[:, 1]

# Grafico desplazamiento
plt.figure(figsize=(8,5))
plt.plot(t, x)
plt.xlabel("Tiempo [s]")
plt.ylabel("Desplazamiento [m]")
plt.title("Respuesta dinamica del sistema")
plt.grid(True)
plt.show()

# Grafico velocidad
plt.figure(figsize=(8,5))
plt.plot(t, v)
plt.xlabel("Tiempo [s]")
plt.ylabel("Velocidad [m/s]")
plt.title("Velocidad de la masa")
plt.grid(True)
plt.show()
```

Implementación general de Heun

Para una ecuación:

$$y' = f(t, y)$$

el algoritmo puede resumirse como:

$$k_1 = f(t_k, Y_k),$$

$$Y_p = Y_k + h k_1,$$

$$k_2 = f(t_k + h, Y_p),$$

$$Y_{k+1} = Y_k + \frac{h}{2}(k_1 + k_2).$$

En Python:

```
k1 = f(t[k], y[k])  
yp = y[k] + h*k1  
k2 = f(t[k] + h, yp)  
y[k+1] = y[k] + h/2*(k1+k2)
```

Ventaja

La misma estructura puede utilizarse para ecuaciones escalares y sistemas de ecuaciones.

Implementación para sistemas

Para:

$$\mathbf{Y}' = \mathbf{F}(t, \mathbf{Y})$$

el algoritmo es:

$$\mathbf{k}_1 = \mathbf{F}(t_k, \mathbf{Y}_k),$$

$$\mathbf{Y}_p = \mathbf{Y}_k + h \mathbf{k}_1,$$

$$\mathbf{k}_2 = \mathbf{F}(t_k + h, \mathbf{Y}_p),$$

$$\mathbf{Y}_{k+1} = \mathbf{Y}_k + \frac{h}{2}(\mathbf{k}_1 + \mathbf{k}_2).$$

Idea fundamental

Una EDO de orden superior puede convertirse en un sistema de EDO de primer orden y resolverse con el mismo algoritmo.

Definimos el error absoluto:

$$E_k = |y(t_k) - Y_k|$$

Si no conocemos la solución exacta, podemos comparar dos soluciones con diferentes pasos.

Para un método de orden p :

$$E(h) \approx Ch^p.$$

Si utilizamos:

$$h_1 = h, \quad h_2 = \frac{h}{2},$$

entonces:

$$\frac{E(h)}{E(h/2)} \approx 2^p.$$

Por tanto, podemos estimar el orden:

$$p \approx \log_2 \left(\frac{E(h)}{E(h/2)} \right)$$

Orden esperado

Para Euler:

$$p = 1$$

y:

$$\frac{E(h)}{E(h/2)} \approx 2.$$

Para Heun:

$$p = 2$$

y:

$$\frac{E(h)}{E(h/2)} \approx 4.$$

Para Taylor de orden 2:

$$p = 2$$

y:

$$\frac{E(h)}{E(h/2)} \approx 4.$$

Interpretación

La reducción del tamaño de paso permite verificar experimentalmente el orden de convergencia del método.

$$Y_{k+1}^{(p)} = Y_k + hf(t_k, Y_k)$$

$$Y_{k+1} = Y_k + \frac{h}{2} \left[f(t_k, Y_k) + f(t_{k+1}, Y_{k+1}^{(p)}) \right]$$

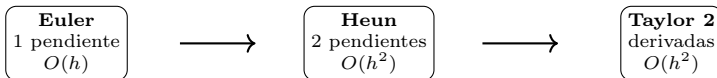
- Predictor: Euler.
- Corrector: promedio de pendientes.
- Método explícito.
- Error global $O(h^2)$.
- No requiere derivadas de f .
- Adecuado para ecuaciones y sistemas de EDO.

Resumen

$$Y_{k+1} = Y_k + hf_k + \frac{h^2}{2}(f_t + f_y f)_k$$

- Se obtiene directamente de Taylor.
- Error global $O(h^2)$.
- Requiere f_t y f_y .
- Puede ser muy preciso para funciones suaves.
- La obtención de derivadas puede ser laboriosa.

Comparación conceptual



Mensaje principal

El aumento del orden de precisión requiere incorporar más información sobre el comportamiento local de la solución.

1. Las EDO permiten modelar numerosos problemas de ingeniería.
2. Euler constituye el punto de partida para comprender los métodos de integración numérica.
3. Heun mejora Euler mediante una estrategia **predictor–corrector**.
4. Heun posee error global:

$$O(h^2).$$

5. Taylor de orden 2 alcanza también:

$$O(h^2),$$

pero requiere derivadas parciales de la función f .

6. Los algoritmos pueden implementarse fácilmente en Python.
7. Los mismos procedimientos pueden utilizarse para sistemas de ecuaciones y problemas de ingeniería.

¿Qué método utilizaría para resolver una EDO de ingeniería cuando:

- $f(t, y)$ es sencilla de evaluar,
- pero sus derivadas f_t y f_y son complicadas?

¿Euler, Heun o Taylor?

¿Preguntas? Método de Euler Mejorado

¿Preguntas?

Métodos Taylor

Métodos de Runge–Kutta

Al finalizar la sesión, el estudiante podrá:

- Formular un problema de valor inicial:

$$y' = f(t, y), \quad y(t_0) = y_0.$$

- Explicar la relación entre Runge–Kutta y la serie de Taylor.
- Aplicar RK2 y RK4.
- Implementar RK4 mediante Python, C y C++.
- Analizar el efecto del tamaño de paso h .
- Resolver aplicaciones de ingeniería mediante EDO.
- Reconocer el método Runge–Kutta–Fehlberg (RK45).

Problema de valor inicial

Consideremos

$$\frac{dy}{dt} = f(t, y)$$

con

$$y(t_0) = y_0$$

La malla numérica es

$$t_k = t_0 + kh, \quad k = 0, 1, \dots, N.$$

A partir de (t_k, Y_k) buscamos una aproximación $Y_{k+1} \approx y(t_{k+1})$.

Idea fundamental

Runge–Kutta obtiene información de la pendiente mediante varias evaluaciones de $f(t, y)$, evitando calcular explícitamente derivadas de orden superior.

Runge–Kutta de Segundo Orden

Serie de Taylor: La expansión de Taylor alrededor de t_k es

$$Y_{k+1} = Y_k + hY'_k + \frac{h^2}{2!}Y''_k + \frac{h^3}{3!}Y'''_k + \cdots$$

Como

$$Y'_k = f(t_k, Y_k),$$

pero

$$Y''_k = \frac{dY'_k}{dt} = \frac{df(t_k, Y_k)}{dt} = f_t + f_y f,$$

el cálculo de derivadas superiores puede ser complicado.

Idea de Runge–Kutta

Obtener una precisión equivalente a un desarrollo de Taylor de orden alto usando únicamente evaluaciones de $f(t, y)$.

La forma general es

$$Y_{k+1} = Y_k + aK_1 + bK_2,$$

donde

$$K_1 = hf(t_k, Y_k),$$

$$K_2 = hf(t_k + \alpha h, Y_k + \beta K_1)$$

Al comparar con Taylor:

$$a + b = 1, \quad b\alpha = \frac{1}{2}, \quad b\beta = \frac{1}{2}.$$

Heun / Euler mejorado

Para

$$a = b = \frac{1}{2}, \quad \alpha = \beta = 1,$$

se obtiene

$$Y_{k+1} = Y_k + \frac{1}{2}(K_1 + K_2)$$

con

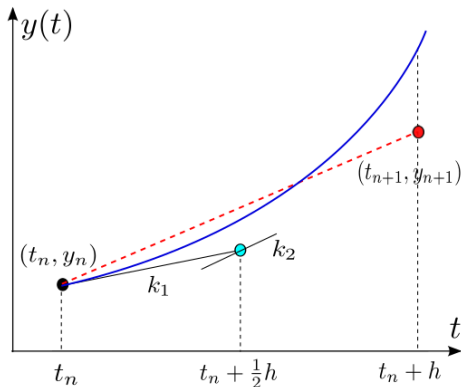
$$K_1 = hf(t_k, Y_k),$$

$$K_2 = hf(t_k + h, Y_k + K_1)$$

Runge–Kutta de Segundo Orden

$$Y_{k+1} = Y_k + \underbrace{\frac{1}{2} \underbrace{f(t_k, Y_k)}_{K_1}}_{K_2} + \underbrace{\frac{1}{2} h \underbrace{f(t_k + h, Y_k + \underbrace{f(t_k, Y_k)h}_{K_1})}_{K_2}}_{K_2} = Y_k + \frac{1}{2} K_1 + \frac{1}{2} K_2$$

Las pendientes intermedias mejoran la aproximación de la pendiente promedio en el intervalo



Runge-Kutta de Cuarto Orden

RK4

El RK4 clásico es

$$Y_{k+1} = Y_k + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4)$$

con

$$K_1 = hf(t_k, Y_k),$$

$$K_2 = hf\left(t_k + \frac{h}{2}, Y_k + \frac{K_1}{2}\right),$$

$$K_3 = hf\left(t_k + \frac{h}{2}, Y_k + \frac{K_2}{2}\right),$$

$$K_4 = hf(t_k + h, Y_k + K_3).$$

Precisión

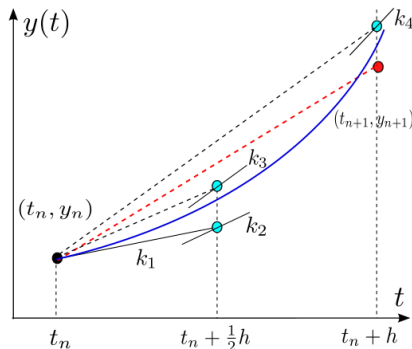
Error local:

$$O(h^5)$$

Error global:

$$O(h^4)$$

Interpretación de las cuatro pendientes



$$Y_{k+1} = Y_k + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4)$$

Las pendientes intermedias mejoran la aproximación de la pendiente promedio en el intervalo.

Algoritmo de RK4

Datos: $f(t, y)$, t_0 , y_0 , t_f y h .

1. Calcular

$$N = \frac{t_f - t_0}{h}.$$

2. Inicializar $t = t_0$, $Y = y_0$.

3. Para $k = 0, \dots, N - 1$:

$$K_1 = hf(t_k, Y_k),$$

$$K_2 = hf(t_k + h/2, Y_k + K_1/2),$$

$$K_3 = hf(t_k + h/2, Y_k + K_2/2),$$

$$K_4 = hf(t_k + h, Y_k + K_3).$$

4. Actualizar:

$$Y_{k+1} = Y_k + \frac{K_1 + 2K_2 + 2K_3 + K_4}{6}.$$

5. Actualizar $t_{k+1} = t_k + h$.

Pseudocódigo RK4

Entrada: $f(t, y)$, t_0 , y_0 , t_f , h

$N = (t_f - t_0)/h$

$t = t_0$

$y = y_0$

para $k = 0, \dots, N-1$:

$K_1 = h \cdot f(t, y)$

$K_2 = h \cdot f(t+h/2, y+K_1/2)$

$K_3 = h \cdot f(t+h/2, y+K_2/2)$

$K_4 = h \cdot f(t+h, y+K_3)$

$y = y + (K_1 + 2 \cdot K_2 + 2 \cdot K_3 + K_4)/6$

$t = t + h$

salida: t, y

Ejemplo 1: RK4 paso a paso

Example

Resolver

$$\frac{dy}{dt} = \frac{2}{t}y + t^2e^t, \quad 1 \leq t \leq 2, \quad y(1) = 0$$

con

$$h = 0,1$$

Definimos

$$f(t, y) = \frac{2}{t}y + t^2e^t.$$

Para $t_0 = 1$, $Y_0 = 0$:

$$\begin{aligned} K_1 &= hf(t_0, Y_0) = 0,1f(1, 0) \\ &= 0,1e \approx 0,27182818 \end{aligned}$$

Luego:

$$\begin{aligned} K_2 &= hf\left(t_0 + \frac{h}{2}, Y_0 + \frac{1}{2}K_1\right) \\ &= 0,1f\left(1,05, \frac{0,27182818}{2}\right) \approx 0,3409444 \end{aligned}$$

$$\begin{aligned} K_3 &= hf\left(t_0 + \frac{h}{2}, Y_0 + \frac{1}{2}K_2\right) \\ &= 0,1f\left(1,05, \frac{0,3409444}{2}\right) \approx 0,3475269 \end{aligned}$$

$$\begin{aligned} K_4 &= hf(t_0 + h, Y_0 + K_3) \\ &= 0,1f(1,1, 0,3475269) \approx 0,4266908 \end{aligned}$$

Actualización y Error

$$Y_1 = Y_0 + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4)$$

Para $t_1 = 1,1$:

$$\begin{aligned} Y_1 &= 0 + \frac{1}{6}(0,27182818 + 2(0,3409444) + \\ &\quad 2(0,3475269) + 0,4266908) = 0,3459103 \end{aligned}$$

Para $t_1 = 1,2$, repitiendo el procedimiento

$$Y_2 \approx 0,8666217$$

La solución exacta es

$$\phi(t) = t^2e^t - e$$

En $t = 1,2$:

$$\phi(1,2) \approx 0,8666425$$

Entonces el error:

$$E = |\phi(1,2) - Y_2| \approx 2,08 \times 10^{-5}$$

Python: ejemplo 1

```
import numpy as np
import matplotlib.pyplot as plt

def f(t, y):
    return 2*y/t + t**2*np.exp(t)

def rk4(f, t0, y0, tf, h):
    n = int(round((tf-t0)/h))
    t = np.zeros(n+1)
    y = np.zeros(n+1)
    t[0], y[0] = t0, y0

    for k in range(n):
        k1 = h*f(t[k], y[k])
        k2 = h*f(t[k]+h/2, y[k]+k1/2)
        k3 = h*f(t[k]+h/2, y[k]+k2/2)
        k4 = h*f(t[k]+h, y[k]+k3)

        y[k+1] = y[k] + (k1+2*k2+2*k3+k4)/6
        t[k+1] = t[k] + h

    return t, y

t, y = rk4(f, 1, 0, 2, 0.1)
exacta = t**2*np.exp(t)-np.e

print(y[2], exacta[2], abs(y[2]-exacta[2]))

plt.plot(t, y, 'o-', label='RK4')
plt.plot(t, exacta, '--', label='Exacta')
plt.xlabel('t');
plt.ylabel('y(t)')
plt.grid();
plt.legend();
plt.show()
```

Ejemplo 2: modelo logístico

Example

Considere

$$\frac{dP}{dt} = rP \left(1 - \frac{P}{K} \right)$$

con

$$r = 0,8,$$

$$K = 1000,$$

$$P(0) = 100,$$

y

$$h = 0,5$$

La función es

$$f(t, P) = 0,8P \left(1 - \frac{P}{1000} \right).$$

Para cada paso:

$$K_1 = hf(t_k, P_k),$$

$$K_2 = hf(t_k + h/2, P_k + K_1/2),$$

$$K_3 = hf(t_k + h/2, P_k + K_2/2),$$

$$K_4 = hf(t_k + h, P_k + K_3),$$

y

$$P_{k+1} = P_k + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4).$$

Solución analítica

Para el modelo logístico:

$$P(t) = \frac{K}{1 + \left(\frac{K - P_0}{P_0} \right) e^{-rt}}.$$

Por tanto,

$$P(t) = \frac{1000}{1 + 9e^{-0,8t}}$$

La solución analítica permite validar el algoritmo numérico.

Interpretación

La población crece inicialmente de forma rápida y luego el crecimiento disminuye cuando P se aproxima a la capacidad de carga K .

Python: ejemplo 2

```
import numpy as np
import matplotlib.pyplot as plt

r, K = 0.8, 1000.0

def f(t, P):
    return r*P*(1-P/K)

def rk4(f, t0, y0, tf, h):
    n = int(round((tf-t0)/h))
    t = np.zeros(n+1)
    y = np.zeros(n+1)
    t[0], y[0] = t0, y0

    for k in range(n):
        k1 = h*f(t[k], y[k])
        k2 = h*f(t[k]+h/2, y[k]+k1/2)
        k3 = h*f(t[k]+h/2, y[k]+k2/2)
        k4 = h*f(t[k]+h, y[k]+k3)
        y[k+1] = y[k] + (k1+2*k2+2*k3+k4)/6
        t[k+1] = t[k] + h

    return t, y

t, P = rk4(f, 0, 100, 5, 0.5)
P_exact = 1000/(1+9*np.exp(-0.8*t))

print("RK4:", P[-1])
print("Exacta:", P_exact[-1])

plt.plot(t, P, 'o-', label='RK4')
plt.plot(t, P_exact, '--', label='Exacta')
plt.xlabel('Tiempo');
plt.ylabel('Poblacion')
plt.grid();
plt.legend();
plt.show()
```

Error y tamaño de paso

Para RK4:

$$E(h) \approx Ch^4.$$

Si se reduce el paso a la mitad:

$$E\left(\frac{1}{2}h\right) \approx \frac{E(h)}{16}.$$

Por tanto:

$$E\left(\frac{1}{2}h\right) \approx \frac{E(h)}{16}.$$

Si se conoce aproximadamente C y se desea $E(h) \leq \varepsilon$:

$$\varepsilon \approx Ch^4 \implies \boxed{h \approx \left(\frac{\varepsilon}{C}\right)^{1/4}}.$$

Sean Y_h y $Y_{h/2}$ las aproximaciones en el mismo punto.

Para RK4:

$$Y_h = \phi + Ch^4,$$

$$Y_{h/2} = \phi + C\frac{h^4}{16}$$

Entonces:

$$Y_{h/2} - Y_h = -\frac{15}{16}Ch^4.$$

Estimación práctica del error

$$|E_h| \approx \frac{16}{15}|Y_{h/2} - Y_h|$$

y para $h/2$

$$|E_{h/2}| \approx \frac{1}{15}|Y_{h/2} - Y_h|$$

Considerando el error local $E_k = Ch^5$ y que con la mitad de h se necesitan evaluar 2 puntos para evaluar k :

$$2E_{k/2} = 2C\frac{h^5}{32}$$

$$E_k - 2E_{k/2} = Ch^5 - 2C\frac{h^5}{32} = \frac{15}{16}Ch^5 \Rightarrow C = \frac{16(E_k - 2E_{k/2})}{15h^5}$$

y

$$\left. \begin{array}{l} y|_k \approx \phi(1) + E_k \\ y|_{k/2} \approx \phi(1) + E_{k/2} \end{array} \right\} \Rightarrow E_k - 2E_{k/2} \approx y|_{k/2} - y|_k \Rightarrow C = \frac{16}{15} \frac{y|_{k/2} - y|_k}{h^5}$$

considerando: $\varepsilon = E_k = Ch^5 \Rightarrow h = \left(\frac{\varepsilon}{C}\right)^{1/5}$

Aplicación 1: movimiento vertical con resistencia

Example

Para un cuerpo bajo gravedad y resistencia lineal:

$$m \frac{dv}{dt} = -mg - cv,$$
$$\frac{dx}{dt} = v.$$

Se obtiene el sistema

$$\begin{cases} x' = v, \\ v' = -g - \frac{c}{m}v. \end{cases}$$

Con

$$x(0) = 0, \quad v(0) = v_0.$$

Definimos

$$\mathbf{Y} = \begin{bmatrix} x \\ v \end{bmatrix}, \quad \mathbf{Y}' = \begin{bmatrix} v \\ -g - (c/m)v \end{bmatrix}.$$

RK4 se aplica componente a componente.

Python: aplicación 1

```
import numpy as np
import matplotlib.pyplot as plt

g, m, c, v0 = 9.81, 2.0, 0.35, 40.0

def F(t, Y):
    x, v = Y
    return np.array([v, -g-(c/m)*v])

def rk4_system(F, t0, Y0, tf, h):
    n = int(round((tf-t0)/h))
    t = np.zeros(n+1)
    Y = np.zeros((n+1, len(Y0)))
    t[0], Y[0] = t0, Y0

    for k in range(n):
        k1 = h*F(t[k], Y[k])
        k2 = h*F(t[k]+h/2, Y[k]+k1/2)
        k3 = h*F(t[k]+h/2, Y[k]+k2/2)
        k4 = h*F(t[k]+h, Y[k]+k3)

        Y[k+1] = Y[k] + (k1+2*k2+2*k3+k4)/6
        t[k+1] = t[k] + h

    return t, Y

t, Y = rk4_system(F, 0, [0, v0], 8, 0.01)
x, v = Y[:,0], Y[:,1]

print("Altura maxima =", np.max(x))

plt.plot(t, x)
plt.xlabel("Tiempo [s]");
plt.ylabel("Altura [m]")
plt.grid();
plt.show()
```

Aplicación 2: enfriamiento de Newton

Example

El modelo es

$$\frac{dT}{dt} = -k(T - T_a)$$

con

$$T(0) = 100^{\circ}\text{C},$$

$$T_a = 20^{\circ}\text{C},$$

$$k = 0,15 \text{ h}^{-1}.$$

La solución exacta es

$$T(t) = T_a + (T_0 - T_a)e^{-kt}.$$

RK4 es especialmente útil cuando el modelo se amplía, por ejemplo:

$$\frac{dT}{dt} = F(t, T),$$

con T_a , k o una fuente térmica dependientes del tiempo.

Python: aplicación 2

```
import numpy as np
import matplotlib.pyplot as plt

Ta, T0, k = 20.0, 100.0, 0.15

def f(t, T):
    return -k*(T-Ta)

def rk4(f, t0, y0, tf, h):
    n = int(round((tf-t0)/h))
    t = np.zeros(n+1)
    y = np.zeros(n+1)
    t[0], y[0] = t0, y0

    for i in range(n):
        k1 = h*f(t[i], y[i])
        k2 = h*f(t[i]+h/2, y[i]+k1/2)
        k3 = h*f(t[i]+h/2, y[i]+k2/2)
        k4 = h*f(t[i]+h, y[i]+k3)/1.0
        y[i+1] = y[i] + (k1+2*k2+2*k3+k4)/6
        t[i+1] = t[i] + h

    return t, y

t, T = rk4(f, 0, T0, 24, 0.25)
T_exact = Ta + (T0-Ta)*np.exp(-k*t)

print("Error maximo =", np.max(abs(T-T_exact)))

plt.plot(t, T, label="RK4")
plt.plot(t, T_exact, "--", label="Exacta")
plt.xlabel("Tiempo [h]"); plt.ylabel("Temperatura [oC]")
plt.grid(); plt.legend(); plt.show()
```

Runge–Kutta–Fehlberg (RKF45)

RKF45 utiliza seis evaluaciones para obtener dos aproximaciones de diferente orden.

$$K_1 = hf(x_k, Y_k),$$

$$K_2 = hf(x_k + \frac{1}{4}h, Y_k + \frac{1}{4}K_1),$$

$$K_3 = hf(x_k + \frac{3}{8}h, Y_k + \frac{3}{32}K_1 + \frac{9}{32}K_2),$$

$$K_4 = hf(x_k + \frac{12}{13}h, Y_k + \frac{1932}{2197}K_1 - \frac{7200}{2197}K_2 + \frac{7296}{2197}K_3),$$

$$K_5 = hf(x_k + h, Y_k + \frac{439}{216}K_1 - 8K_2 + \frac{3680}{513}K_3 - \frac{845}{4104}K_4),$$

$$K_6 = hf(x_k + \frac{1}{2}h, Y_k + \frac{8}{27}K_1 + 2K_2 + \frac{3544}{2565}K_3 + \frac{1859}{4104}K_4 - \frac{11}{40}K_5).$$

RKF45: aproximaciones de orden 4 y 5

Aproximación de orden 4:

$$Y_{k+1}^{(4)} = Y_k + \frac{25}{216}K_1 + \frac{1408}{2565}K_3 + \frac{2197}{4104}K_4 - \frac{1}{5}K_5.$$

Aproximación de orden 5:

$$Y_{k+1}^{(5)} = Y_k + \frac{16}{135}K_1 + \frac{6656}{12825}K_3 + \frac{28561}{56430}K_4 - \frac{9}{50}K_5 + \frac{2}{55}K_6.$$

Estimación:

$$E_k = Y_{k+1}^{(5)} - Y_{k+1}^{(4)}$$

Nota

Los coeficientes deben escribirse como fracciones exactas; por ejemplo 12825 y 28561/56430, no como números decimales truncados.

Control adaptativo del paso

Una estrategia práctica es

$$h_{\text{nuevo}} = 0,84 h \left(\frac{\varepsilon}{|E_k|} \right)^{1/5}.$$

Procedimiento:

1. Calcular $Y^{(4)}$ y $Y^{(5)}$.
2. Estimar E_k .
3. Si $|E_k| \leq \varepsilon$, aceptar el paso.
4. Si $|E_k| > \varepsilon$, rechazarlo.
5. Actualizar h y repetir.

Ventaja

El algoritmo puede utilizar pasos pequeños donde la solución cambia rápidamente y pasos grandes donde la solución es suave.

Función RK4 reutilizable

```
def rk4(f, t0, y0, tf, h):  
    n = int(round((tf-t0)/h))  
    t = np.zeros(n+1)  
    y = np.zeros(n+1)  
  
    t[0], y[0] = t0, y0  
  
    for k in range(n):  
        k1 = h*f(t[k], y[k])  
        k2 = h*f(t[k]+h/2, y[k]+k1/2)  
        k3 = h*f(t[k]+h/2, y[k]+k2/2)  
        k4 = h*f(t[k]+h, y[k]+k3)  
  
        y[k+1] = y[k] + (k1+2*k2+2*k3+k4)/6  
        t[k+1] = t[k] + h  
  
    return t, y
```

Idea

Una sola función permite reutilizar el algoritmo para múltiples EDO.

RK4 para sistemas

Para

$$\begin{aligned}\mathbf{Y}' &= \mathbf{F}(t, \mathbf{Y}), & \mathbf{Y}(t_0) &= \mathbf{Y}_0, \\ \mathbf{K}_1 &= h\mathbf{F}(t_k, \mathbf{Y}_k), \\ \mathbf{K}_2 &= h\mathbf{F}(t_k + h/2, \mathbf{Y}_k + \mathbf{K}_1/2), \\ \mathbf{K}_3 &= h\mathbf{F}(t_k + h/2, \mathbf{Y}_k + \mathbf{K}_2/2), \\ \mathbf{K}_4 &= h\mathbf{F}(t_k + h, \mathbf{Y}_k + \mathbf{K}_3), \\ \mathbf{Y}_{k+1} &= \mathbf{Y}_k + \frac{\mathbf{K}_1 + 2\mathbf{K}_2 + 2\mathbf{K}_3 + \mathbf{K}_4}{6}.\end{aligned}$$

Aplicación

La misma formulación permite resolver sistemas de EDO de primer orden y ecuaciones de orden superior transformadas en sistemas.

Comparación de métodos

Método	Evaluaciones/paso	Error global	Característica
Euler	1	$O(h)$	Muy simple
RK2	2	$O(h^2)$	Mejor precisión
RK4	4	$O(h^4)$	Alta precisión
RKF45	6	Adaptativo	Control de h

Criterio práctico

RK4 ofrece una excelente relación entre costo computacional y precisión para numerosos problemas de ingeniería.

Ventajas

- Alta precisión.
- No requiere derivadas superiores.
- Implementación sencilla.
- Funciona con sistemas.
- Amplia aplicación en ingeniería.

Limitaciones

- Varias evaluaciones de f .
- RK4 clásico usa paso fijo.
- Puede ser ineficiente en problemas rígidos.
- En problemas exigentes conviene usar control adaptativo.

Conclusiones

1. Runge–Kutta obtiene alta precisión sin calcular derivadas superiores.
2. RK4 utiliza cuatro evaluaciones de f por paso.
3. Su error global es $O(h^4)$.
4. Puede aplicarse directamente a sistemas de EDO.
5. Python permite automatizar cálculos, gráficos y análisis de error.
6. RKF45 permite estimar el error y adaptar h .

La elección de h es tan importante como la elección del método.

Ejercicio propuesto

Resolver mediante RK4 con $h = 0,1$:

$$y' = y - t^2 + 1, \quad y(0) = 0,5, \quad 0 \leq t \leq 2.$$

1. Calcular manualmente K_1, K_2, K_3, K_4 para el primer paso.
2. Obtener Y_1 y Y_2 .
3. Implementar el algoritmo en Python.
4. Comparar con la solución exacta.
5. Repetir con $h = 0,05$.
6. Verificar:

$$\frac{E(h)}{E(h/2)} \approx 16.$$

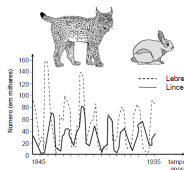
¿Preguntas?

Métodos Runge Kutta

Sistema de Ecuacion Diferenciales

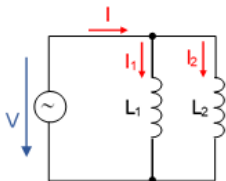
Sistemas de EDOs de primer orden

Modelo predador-presa

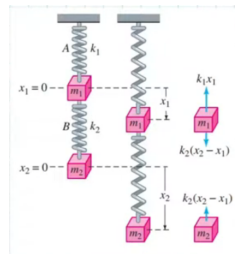


$$\begin{aligned}\frac{dx}{dt} &= Ax - Bxy \\ \frac{dy}{dt} &= -Cx + Dxy\end{aligned}$$

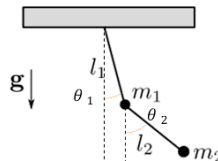
Movimiento vertical de los resortes acoplados



Corrientes en una red eléctrica



Péndulo doble



Ejemplo 1: Sistema de dos tanques

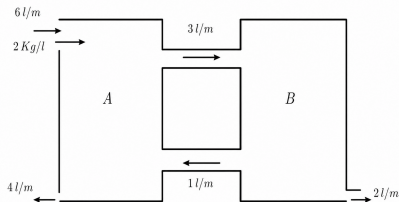
Dos tanques, A y B , contienen inicialmente 100 L de líquido cada uno y se encuentran interconectados mediante tuberías.

- Del tanque A al tanque B fluyen 3 L/min.
- Del tanque B al tanque A fluye 1 L/min.
- Una salmuera con concentración 2 kg/L ingresa al tanque A a razón de 6 L/min.
- Del tanque A salen 4 L/min hacia el exterior.
- Del tanque B salen 2 L/min hacia el exterior.

Se supone que el contenido de ambos tanques permanece perfectamente mezclado. Inicialmente,

$$A(0) = 0 \text{ kg}, \quad B(0) = 200 \text{ kg}.$$

Se desea determinar la cantidad de sal en cada tanque durante los primeros tres minutos utilizando el método de Euler.



- $A(t)$, concentración (kg) de sal en el tanque A en t minutos
- $B(t)$, concentración (kg) de sal en el tanque B en t minutos

Ejemplo 1: Formulación del modelo

Tanque A

La cantidad de sal satisface

$$\frac{dA}{dt} = \text{Entrada} - \text{Salida}.$$

Entrada:

$$2 \frac{\text{kg}}{\text{L}}(6) + \frac{B}{100}(1) = 12 + \frac{B}{100} \quad \frac{\text{kg}}{\text{min}}$$

Salida:

$$\frac{A}{100}(4) + \frac{A}{100}(3) = \frac{7A}{100} \quad \frac{\text{kg}}{\text{min}}$$

Por tanto,

$$\frac{dA}{dt} = 12 + 0,01B - 0,07A$$

Tanque B

Entrada:

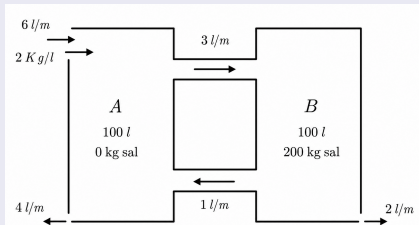
$$\frac{A}{100}(3) = \frac{3A}{100}$$

Salida:

$$\frac{B}{100}(1) + \frac{B}{100}(2) = \frac{3B}{100}$$

Por tanto,

$$\frac{dB}{dt} = 0,03A - 0,03B$$



Ejemplo 1: Método de Euler

El sistema queda

$$\frac{dA}{dt} = 12 + 0,01B - 0,07A,$$

$$\frac{dB}{dt} = 0,03A - 0,03B.$$

Se sabe que inicialmente el tanque A contenía agua pura y el B contiene 200 kg de sal, por lo tanto:

$$A(0) = 0, \quad B(0) = 200.$$

Para un paso $h = 1$ min, el método de Euler es

$$A_{k+1} = A_k + hf(t_k, A_k, B_k)$$

$$B_{k+1} = B_k + hf(t_k, A_k, B_k)$$

Entonces

$$A_{k+1} = A_k + h(12 + 0,01B_k - 0,07A_k),$$

$$B_{k+1} = B_k + h(0,03A_k - 0,03B_k).$$

$$A_1 = 0 + (1)(12 + 0,01 B_0 - 0,07 A_0)$$

$$= 12 + (0,01)(200) = 14$$

$$A_2 = 14 + (1)(12 + 0,01 B_1 - 0,07 A_1)$$

$$= 14 + 12 + (0,01)(194) - (0,07)(14) = 26,96$$

$$A_3 = 26,96 + (1)(12 + 0,01 B_2 - 0,07 A_2)$$

$$= 26,96 + 12 + (0,01)(188,6 - (0,07)(26,96)) = 38,953$$

$$B_1 = 200 + (1)(0,03 A_0 - 0,03 B_0)$$

$$= 200 - (0,03)(200) = 194$$

$$B_2 = 194 + (1)(0,03 A_1 - 0,03 B_1)$$

$$= 194 + (0,03)(14) - (0,03)(194) = 188,6$$

$$B_3 = 188,6 + (1)(0,03 A_2 - 0,03 B_2)$$

$$= 188,6 + (0,03)(26,96) - (0,03)(188,6) = 183,75$$

t (min)	$A(t)$ (kg)	$B(t)$ (kg)
0	0,000	200,000
1	14,000	194,000
2	26,960	188,600
3	38,953	183,750

Resultado

Después de tres minutos,

$$A(3) \approx 38,95 \text{ kg},$$

$$B(3) \approx 183,75 \text{ kg}.$$

Ejemplo 1 -Solución

Tiempo t min	Cantidad de sal $A(t)$ kg	Cantidad de sal $B(t)$ kg	Concentración $C_A(t)$ kg/L	Concentración $C_B(t)$ kg/L
0	0.0000	200.0000	0.0000	2.0000
1	14.0000	194.0000	0.1400	1.9400
2	26.9600	188.6000	0.2696	1.8860
3	38.9588	183.7508	0.3896	1.8375

Resolver:

$$\frac{dx}{dt} = x' = f_1(t, x, y)$$

$$\frac{dy}{dt} = x' = f_2(t, x, y)$$

con condiciones iniciales

$$x(t_0) = x_0 \quad y(t_0) = y_0$$

y con paso h

Se aplica algún método (preferiblemente Runge-Kutta de orden cuatro) a cada una de las ecuaciones del sistema.

Runge-Kutta de 4^{to} orden

Paso 1: Calcular las pendientes iniciales al inicio del intervalo

$$K_1 = hf_1(t_k, X_k, Y_k)$$

$$L_1 = hf_2(t_k, X_k, Y_k)$$

Paso 2: Calcular las pendientes en el punto medio usando K_1 y L_1

$$K_2 = hf_1\left(t_k + \frac{1}{2}h, X_k + \frac{1}{2}K_1, Y_k + \frac{1}{2}L_1\right)$$

$$L_2 = hf_2\left(t_k + \frac{1}{2}h, X_k + \frac{1}{2}K_1, Y_k + \frac{1}{2}L_1\right)$$

Paso 3: Recalcular las pendientes en el punto medio usando K_2 y L_2

$$K_3 = hf_1\left(t_k + \frac{1}{2}h, X_k + \frac{1}{2}K_2, Y_k + \frac{1}{2}L_2\right)$$

$$L_3 = hf_2\left(t_k + \frac{1}{2}h, X_k + \frac{1}{2}K_2, Y_k + \frac{1}{2}L_2\right)$$

Paso 4: Recalcular las pendientes en el punto medio usando K_3 y L_3

$$K_4 = hf_1(t_k + h, X_k + K_3, Y_k + L_3)$$

$$L_4 = hf_2(t_k + h, X_k + K_3, Y_k + L_3)$$

Paso 5: Actualizar las variables

$$X_{k+1} = X_k + \frac{1}{6}[K_1 + 2K_2 + 2K_3 + K_4]$$

$$Y_{k+1} = Y_k + \frac{1}{6}[L_1 + 2L_2 + 2L_3 + L_4]$$

Ejemplo - Sistemas de EDOs (RK4)

Resolver:

$$\begin{cases} \frac{dx}{dt} = x + 2y = f(t, x, y) \\ \frac{dy}{dt} = 3x + 2y = g(t, x, y) \\ x(0) = 6, \quad y(0) = 4 \end{cases}$$

con paso $h = 0,02$

Paso $k = 0$: $(t_0, x_0, y_0) = (0, 6, 4)$

Cálculo de coeficientes:

$$K_1 = hf(t_0, x_0, y_0) = 0,02(6 + 2 \cdot 4) = 0,28$$

$$L_1 = hg(t_0, x_0, y_0) = 0,02(3 \cdot 6 + 2 \cdot 4) = 0,52$$

$$\left(t_0 + \frac{h}{2}, x_0 + \frac{K_1}{2}, y_0 + \frac{L_1}{2}\right) = (0,01, 6 + 0,14, 4 + 0,26)$$

$$K_2 = 0,02[(6 + 0,14) + 2(4 + 0,26)] = 0,2932$$

$$L_2 = 0,02[3(6 + 0,14) + 2(4 + 0,26)] = 0,5388$$

$$K_3 = 0,02[(6 + 0,1466) + 2(4 + 0,2694)] = 0,2937$$

$$L_3 = 0,02[3(6 + 0,1466) + 2(4 + 0,2694)] = 0,5396$$

$$K_4 = 0,02[(6 + 0,2937) + 2(4 + 0,5396)] = 0,3075$$

$$L_4 = 0,02[3(6 + 0,2937) + 2(4 + 0,5396)] = 0,5592$$

Actualización:

$$x_1 = x_0 + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4) = 6 + \frac{1}{6}(0,28 + 2(0,2932) + 2(0,2937) + 0,3075) \approx 6,2936$$

$$y_1 = y_0 + \frac{1}{6}(L_1 + 2L_2 + 2L_3 + L_4) = 4 + \frac{1}{6}(0,52 + 2(0,5388) + 2(0,5396) + 0,5592) \approx 4,5393$$

$$t_1 = t_0 + h = 0,02$$

Ejemplo - Sistemas de EDOs (RK4)

Paso $k = 1$: $(t_1, x_1, y_1) = (0,02, 6,2936, 4,5393)$

Cálculo de coeficientes:

$$K_1 = 0,02(6,2936 + 2 \cdot 4,5393) = 0,3074$$

$$L_1 = 0,02(3 \cdot 6,2936 + 2 \cdot 4,5393) = 0,5590$$

$$K_3 = 0,02[(6,2936 + 0,1609) + 2(4,5393 + 0,2898)]$$

$$= 0,02(6,4545 + 2 \cdot 4,8291) = 0,3223$$

$$\left(t_1 + \frac{h}{2}, x_1 + \frac{K_1}{2}, y_1 + \frac{L_1}{2}\right) = (0,03, 6,2936+0,1609, 4,5393+0,2898)$$

$$L_3 = 0,02[3(6,2936 + 0,1609) + 2(4,5393 + 0,2898)]$$

$$= 0,02(19,3635 + 9,6582) = 0,5804$$

$$K_2 = 0,02[(6,2936 + 0,1537) + 2(4,5393 + 0,2795)]$$

$$= 0,02(6,4473 + 2 \cdot 4,8188) = 0,3217$$

$$K_4 = 0,02[(6,2936 + 0,3223) + 2(4,5393 + 0,5804)]$$

$$= 0,02(6,6159 + 2 \cdot 5,1197) = 0,3371$$

$$L_2 = 0,02[3(6,2936 + 0,1537) + 2(4,5393 + 0,2795)]$$

$$= 0,02(19,3419 + 9,6376) = 0,5796$$

$$L_4 = 0,02[3(6,2936 + 0,3223) + 2(4,5393 + 0,5804)]$$

$$= 0,02(19,8477 + 10,2394) = 0,6017$$

Actualización:

$$x_2 = x_1 + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4) = 6,2936 + \frac{1}{6}(0,3074 + 2(0,3217) + 2(0,3223) + 0,3371) \approx 6,6158$$

$$y_2 = y_1 + \frac{1}{6}(L_1 + 2L_2 + 2L_3 + L_4) = 4,5393 + \frac{1}{6}(0,5590 + 2(0,5796) + 2(0,5804) + 0,6017) \approx 5,1196$$

$$t_2 = t_1 + h = 0,04$$

Paso $k = 2$: $(t_2, x_2, y_2) = (0,04, 6,6158, 5,1196) \dots$

EDOs de orden N

Si la ecuación es de orden mayor o igual a 2, se transforma la ecuación en un sistema de EDOs de primer orden.

Ejemplo:

$$\begin{cases} t^2 y'' - 2ty' + 2y = t^3 \ln t \\ y(1) = 1, y'(1) = 0, \quad 1 \leq t \leq 1,5 \end{cases}$$

Se transforma el PVI en un sistema de EDOs de primer orden. Se despeja el diferencial de mayor orden. Se reemplazan las funciones por otras equivalentes con el fin de eliminar las diferenciales de orden superior:

$$\begin{cases} y' = g = f_1(t, y, g) \\ g' = y'' = \frac{2}{t}g - \frac{2}{t^2}y + t \ln t = f_2(t, y, g) \\ y(1) = 1, \quad g(1) = 0 \\ h = 0,05, \quad 1 \leq t \leq 1,5 \end{cases}$$

Ejemplo - Sistemas de EDOs (RK4)

$$\begin{cases} y' = g = f_1(t, y, g) \\ g' = y'' = \frac{2}{t}g - \frac{2}{t^2}y + t \ln t = f_2(t, y, g) \\ y(1) = 1, \quad g(1) = 0 \\ h = 0,05, \quad 1 \leq t \leq 1,5 \end{cases}$$

Paso $k = 0$: $(t_0, y_0 = y(1), g_0 = g(1)) = (1, 1, 0)$

Cálculo de coeficientes:

$$K_1 = hf_1(t_0, y_0, g_0) = 0,05(0) = 0$$

$$L_1 = hf_2(t_0, y_0, g_0) = 0,05 \left(\frac{2}{1}(0) - \frac{2}{1^2}(1) + (1) \ln(1) \right) = -0,1$$

$$\left(t_0 + \frac{h}{2}, y_0 + \frac{K_1}{2}, g_0 + \frac{L_1}{2} \right) = (0,0125, 1, -0,05)$$

$$\begin{aligned} K_2 &= 0,05f_1 \left(t_0 + \frac{h}{2}, y_0 + \frac{K_1}{2}, g_0 + \frac{L_1}{2} \right) \\ &= 0,05f_1(0,0125, 1, -0,05) = -2,5 \times 10^{-3} \end{aligned}$$

$$\begin{aligned} L_2 &= 0,05f_2 \left(t_0 + \frac{h}{2}, y_0 + \frac{K_1}{2}, g_0 + \frac{L_1}{2} \right) \\ &= 0,05f_2(0,0125, 1, -0,05) \\ &= 0,05 \left(\frac{2}{1,025}(-0,05) - \frac{2}{1,025^2}(1) + 1,025 \ln(1,025) \right) \\ &\approx -0,98793992 \end{aligned}$$

$$\begin{aligned} K_3 &= 0,05f_1 \left(t_0 + \frac{h}{2}, y_0 + \frac{K_1}{2}, g_0 + \frac{L_1}{2} \right) \\ &= 0,05f_1(0,0125, 1, -0,05) = -2,5 \times 10^{-3} \end{aligned}$$

$$\begin{aligned} L_3 &= 0,05f_2 \left(t_0 + \frac{h}{2}, y_0 + \frac{K_1}{2}, g_0 + \frac{L_1}{2} \right) \\ &= 0,05f_2(0,0125, 1, -0,05) = -0,98793992 \end{aligned}$$

$$\begin{aligned} K_4 &= 0,05f_1 \left(t_0 + h, y_0 + K_1, g_0 + L_1 \right) \\ &= 0,05f_1(1,05, 0,9975301502, -0,985161855) \\ &= -4,930809278 \times 10^{-3} \end{aligned}$$

$$\begin{aligned} L_4 &= 0,05f_2 \left(t_0 + h, y_0 + K_1, g_0 + L_1 \right) \\ &= 0,05f_2(1,05, 0,9975301502, -0,985161855) \\ &= -0,973094592 \end{aligned}$$

Ejemplo - Sistemas de EDOs (RK4)

Actualización:

$$\begin{aligned}y_1 &= y_0 + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4) \\&= 1 + \frac{1}{6}(0 + 2(-2,5 \times 10^{-3}) + 2(-2,4698498 \times 10^{-3}) - 4,930809278 \times 10^{-3}) \approx 0,9975215819 \\g_1 &= g_0 + \frac{1}{6}(L_1 + 2L_2 + 2L_3 + L_4) \\&= 0 + \frac{1}{6}(-0,1 + 2(-0,98793992) + 2(-0,0986161855) - 0,973094592) \approx -0,0986883023 \\t_1 &= t_0 + h = 1,05\end{aligned}$$

Ejemplo - Sistemas de EDOs (RK4)

Paso $k = 1$: $(t_1, y_1, g_1) = (1,05, 0,9975215819, -0,098688302)$

Cálculo de coeficientes:

$$K_1 = hf_1(t_1, y_1, g_1) = 0,05(-0,098688302) = -0,00493$$

$$\begin{aligned} L_1 &= hf_2(t_1, y_1, g_1) \\ &= 0,05 \left(\frac{2}{1,05}(-0,09869) - \frac{2}{1,05^2}(0,99752) + 1,05 \ln(1,05) \right) \\ &\approx -0,09730 \end{aligned}$$

$$\left(t_1 + \frac{h}{2}, y_1 + \frac{K_1}{2}, g_1 + \frac{L_1}{2} \right) = (1,075, 0,99505, -0,14734)$$

$$K_2 = 0,05f_1\left(t_1 + \frac{h}{2}, y_1 + \frac{K_1}{2}, g_1 + \frac{L_1}{2}\right)$$

$$= 0,05(-0,14734) = -0,00737$$

$$L_2 = 0,05f_2\left(t_1 + \frac{h}{2}, y_1 + \frac{K_1}{2}, g_1 + \frac{L_1}{2}\right)$$

$$\approx -0,09595$$

Actualización:

$$y_2 = y_1 + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4) = 0,99752 + \frac{1}{6}(-0,00493 + 2(-0,00737) + 2(-0,00733) - 0,00972) \approx 0,99020$$

$$g_2 = g_1 + \frac{1}{6}(L_1 + 2L_2 + 2L_3 + L_4) = -0,09869 + \frac{1}{6}(-0,09730 + 2(-0,09595) + 2(-0,09578) - 0,09420) \approx -0,19443$$

$$t_2 = t_1 + h = 1,10$$

$$K_3 = 0,05f_1\left(t_1 + \frac{h}{2}, y_1 + \frac{K_1}{2}, g_1 + \frac{L_1}{2}\right)$$

$$= 0,05(-0,14666) = -0,00733$$

$$L_3 = 0,05f_2\left(t_1 + \frac{h}{2}, y_1 + \frac{K_1}{2}, g_1 + \frac{L_1}{2}\right)$$

$$\approx -0,09578$$

$$K_4 = 0,05f_1\left(t_1 + \frac{h}{2}, y_1 + \frac{K_1}{2}, g_1 + \frac{L_1}{2}\right)$$

$$= 0,05(-0,19447) = -0,00972$$

$$L_4 = 0,05f_2\left(t_1 + \frac{h}{2}, y_1 + \frac{K_1}{2}, g_1 + \frac{L_1}{2}\right)$$

$$\approx -0,09420$$

Código

```
#include <stdio.h>
#include <math.h>

// f1 = y' = g
double f1(double t, double y, double g) {
    return g;
}

// f2 = g' = y''
double f2(double t, double y, double g) {
    return (2.0/t)*g - (2.0/(t*t))*y + t*log(t);
}

int main() {
    double t = 1.0, y = 1.0, g = 0.0;
    double h = 0.05;
    int n = (1.5 - 1.0)/h;

    printf("t\tty\ttg\n");
    printf("%.2f\t%.6f\t%.6f\n", t, y, g);

    for(int i = 0; i < n; i++) {
        // RK4 coeficientes
        double k1 = h * f1(t, y, g);
        double l1 = h * f2(t, y, g);
        double k2 = h * f1(t + h/2, y + k1/2, g + l1/2);
        double l2 = h * f2(t + h/2, y + k1/2, g + l1/2);
        double k3 = h * f1(t + h/2, y + k2/2, g + l2/2);
        double l3 = h * f2(t + h/2, y + k2/2, g + l2/2);
        double k4 = h * f1(t + h, y + k3, g + l3);
        double l4 = h * f2(t + h, y + k3, g + l3);

        // actualizacion
        y = y + (k1 + 2*k2 + 2*k3 + k4)/6.0;
        g = g + (l1 + 2*l2 + 2*l3 + l4)/6.0;
        t = t + h;
        printf("%.2f\t%.6f\t%.6f\n", t, y, g);
    }
    return 0;
}
```

```
import math

# f1 = y' = g
def f1(t, y, g):
    return g

# f2 = g' = y''
def f2(t, y, g):
    return (2.0/t)*g - (2.0/(t**2))*y +
           t*math.log(t)

# condiciones iniciales
t = 1.0
y = 1.0
g = 0.0

h = 0.05
n = int((1.5 - 1.0)/h)

print("t\tty\ttg")
print(f"{t:.2f}\t{y:.6f}\t{g:.6f}")

for i in range(n):
    # coeficientes RK4
    k1 = h * f1(t, y, g)
    l1 = h * f2(t, y, g)
    k2 = h * f1(t + h/2, y + k1/2, g + l1/2)
    l2 = h * f2(t + h/2, y + k1/2, g + l1/2)
    k3 = h * f1(t + h/2, y + k2/2, g + l2/2)
    l3 = h * f2(t + h/2, y + k2/2, g + l2/2)
    k4 = h * f1(t + h, y + k3, g + l3)
    l4 = h * f2(t + h, y + k3, g + l3)

    # actualizacion
    y = y + (k1 + 2*k2 + 2*k3 + k4)/6.0
    g = g + (l1 + 2*l2 + 2*l3 + l4)/6.0
    t = t + h
    print(f"{t:.2f}\t{y:.6f}\t{g:.6f}")
```

ODE Software Packages: BOOTS (C++) and SCIPY (Python)

```
#include <iostream>
#include <vector>
#include <cmath>
#include <boost/numeric/odeint.hpp>
using namespace std;
using namespace boost::numeric::odeint;
// Definicion del tipo de estado [y, g]
typedef vector<double> state_type;
// Sistema de ED0s
void sistema(const state_type &Y, state_type &dYdt,
double t) {
    double y = Y[0];
    double g = Y[1];
    dYdt[0] = g;
    dYdt[1] = (2.0/t)*g - (2.0/(t*t))*y + t*log(t);
}

int main() {
    state_type Y(2);
    Y[0] = 1.0; // y(1)
    Y[1] = 0.0; // g(1)
    double t0 = 1.0;
    double tf = 1.5;
    double h = 0.05;

    // Metodo RK45 (Dormand-Prince)
    runge_kutta_dopri5<state_type> stepper;
    cout << "t\tty\ttg" << endl;
    cout << t0 << "\t" << Y[0] << "\t" << Y[1] << endl;

    double t = t0;
    while (t < tf) {
        stepper.do_step(sistema, Y, t, h);
        t += h;
        cout << t << "\t" << Y[0] << "\t" << Y[1] << endl;
    }
    return 0;
}
```

```
import numpy as np
from scipy.integrate import solve_ivp

# Definicion del sistema
# Y = [y, g]
def sistema(t, Y):
    y, g = Y
    dy_dt = g
    dg_dt = (2/t)*g - (2/(t**2))*y +
            t*np.log(t)
    return [dy_dt, dg_dt]

# Condiciones iniciales
t0 = 1.0
tf = 1.5
Y0 = [1.0, 0.0]

# Puntos donde evaluar la solucion
t_eval = np.arange(t0, tf + 0.05, 0.05)

# Resolver con RK45
sol = solve_ivp(sistema, [t0, tf], Y0,
                method='RK45', t_eval=t_eval)

# Mostrar resultados
print("t\tty\ttg")
for t, y, g in zip(sol.t, sol.y[0], sol.y[1]):
    print(f"{t:.2f}\t{y:.6f}\t{g:.6f}")
```

EDO de Segundo Orden

$$\begin{cases} x'' = f(t, x, x') & \text{para } a \leq t \leq b \\ x(a) = \alpha \wedge x(b) = \beta \end{cases}$$

Son **EDOs de segundo orden**, donde se conocen los valores de la función en los extremos del intervalo (**condiciones de frontera**).

Ejemplo:

Problemas de deflexión de una viga con los extremos sujetos

Posible Solución (Conversión): Se convierte una EDO de segundo orden

$$x'' = f(t, x, x') \quad x(a) = \alpha, \quad x(b) = \beta \quad (\text{condiciones de frontera})$$

en un sistema de primer orden. El problema es que se desconoce la derivada en el valor inicial.

EDO sistema de primer orden

$$\begin{cases} x' = y & \text{falta el valor de } y(a) \\ y' = f(t, x, y) \end{cases}$$

Método del Disparo

Problemas de valores en la frontera

Un problema de valores en la frontera (PVF) para una EDO de segundo orden puede escribirse como

$$y'' = f(x, y, y'), \quad a \leq x \leq b,$$

con condiciones

$$y(a) = \alpha, \quad y(b) = \beta.$$

Dificultad principal

En un PVF se conocen condiciones en **dos puntos diferentes** del dominio. Por ejemplo,

$$y(0) = 0, \quad y(1) = 2.$$

Sin embargo, para resolver una EDO mediante un método de valores iniciales necesitamos conocer

$$y(a), \quad y'(a).$$

Por tanto, la pendiente inicial $y'(a)$ es desconocida.

Idea fundamental del método del disparo

El método del disparo transforma el PVF en una sucesión de **problemas de valores iniciales (PVI)**.

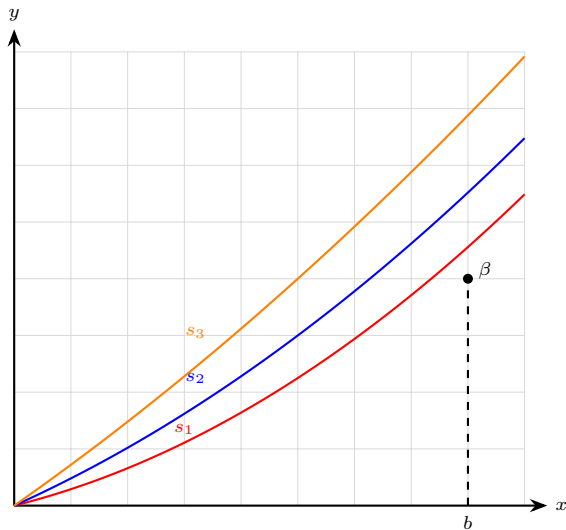
1. Se propone una pendiente inicial s .
2. Se resuelve el PVI desde $x = a$ hasta $x = b$.
3. Se calcula el valor obtenido $y(b; s)$.
4. Se compara con el valor requerido β .
5. Se modifica s .
6. Se repite hasta satisfacer la condición de frontera.

Interpretación

La pendiente $s = y'(a)$ es el “ángulo de disparo”. Cada elección de s produce una trayectoria diferente. Se busca la trayectoria que llegue exactamente al punto final (b, β) .

$$y'(a) = s \quad \longrightarrow \quad y(b; s) \approx \beta$$

Interpretación gráfica



Se modifica la pendiente inicial hasta que la trayectoria alcanza el valor de frontera deseado.

Conversión de la EDO a un sistema de primer orden

Consideremos

$$y'' = f(x, y, y').$$

Definimos

$$z = y'.$$

Entonces

$$y' = z,$$

y

$$z' = f(x, y, z).$$

Por tanto,

$$\begin{cases} y' = z, \\ z' = f(x, y, z). \end{cases}$$

El PVF

$$y(a) = \alpha, \quad y(b) = \beta$$

se transforma en el PVI

$$\begin{cases} y' = z, \\ z' = f(x, y, z), \\ y(a) = \alpha, \\ z(a) = s. \end{cases}$$

donde s es la pendiente desconocida.

Función de disparo

Definimos la función

$$F(s) = y(b; s) - \beta.$$

La solución del PVF corresponde a encontrar

$$F(s) = 0.$$

Es decir,

$$y(b; s) = \beta.$$

Por tanto, el método del disparo puede interpretarse como un problema de búsqueda de raíces.

Problema original

$$y'' = f(x, y, y'), \quad y(a) = \alpha, \quad y(b) = \beta.$$

Problema equivalente

$$F(s) = y(b; s) - \beta = 0.$$

Así, pueden utilizarse métodos como:

Bisección, secante, Newton o disparo lineal

Disparo lineal

Se seleccionan dos pendientes iniciales

$$s_1, \quad s_2.$$

Se resuelven los dos PVI:

$$y_1(b) = y(b; s_1), \quad y_2(b) = y(b; s_2).$$

La pendiente corregida se obtiene mediante interpolación lineal:

$$s_3 = s_1 + \frac{\beta - y_1(b)}{y_2(b) - y_1(b)}(s_2 - s_1)$$

También puede escribirse como

$$s_3 = s_1 - \frac{F(s_1)}{F(s_2) - F(s_1)}(s_2 - s_1).$$

Objetivo

Repetir el procedimiento hasta que

$$|y(b; s_k) - \beta| < \varepsilon.$$

Algoritmo del método del disparo lineal

1. Definir $f(x, y, y')$, a , b , α , β .
2. Elegir dos pendientes iniciales:

$$s_1, \quad s_2.$$

3. Resolver

$$y' = z, \quad z' = f(x, y, z)$$

con

$$y(a) = \alpha, \quad z(a) = s_1.$$

4. Obtener $Y_1 = y(b; s_1)$.
5. Resolver nuevamente con

$$y(a) = \alpha, \quad z(a) = s_2.$$

6. Obtener $Y_2 = y(b; s_2)$.
7. Calcular

$$s_3 = s_1 + \frac{\beta - Y_1}{Y_2 - Y_1}(s_2 - s_1).$$

8. Repetir hasta cumplir la tolerancia.

Pseudocódigo–Método del disparo lineal

Entrada:

```
a, b, alpha, beta  
s1, s2  
tolerancia  
N = numero de pasos
```

Mientras no converja:

```
Resolver PVI con pendiente s1  
Y1 = y(b)
```

```
Resolver PVI con pendiente s2  
Y2 = y(b)
```

```
s3 = s1 + (beta-Y1)/(Y2-Y1)*(s2-s1)
```

```
Resolver PVI con pendiente s3  
Y3 = y(b)
```

```
error = |Y3-beta|
```

```
s1 = s2  
s2 = s3
```

Retornar $y(x)$, $z(x)$, $s3$

Integración del PVI mediante RK4

Para el sistema

$$\mathbf{Y}' = \mathbf{F}(x, \mathbf{Y}), \quad \mathbf{Y} = \begin{bmatrix} y \\ z \end{bmatrix},$$

el método RK4 utiliza

$$\mathbf{k}_1 = \mathbf{F}(x_n, \mathbf{Y}_n),$$

$$\mathbf{k}_2 = \mathbf{F}\left(x_n + \frac{h}{2}, \mathbf{Y}_n + \frac{h}{2}\mathbf{k}_1\right),$$

$$\mathbf{k}_3 = \mathbf{F}\left(x_n + \frac{h}{2}, \mathbf{Y}_n + \frac{h}{2}\mathbf{k}_2\right),$$

$$\mathbf{k}_4 = \mathbf{F}(x_n + h, \mathbf{Y}_n + h\mathbf{k}_3)$$

Finalmente,

$$\mathbf{Y}_{n+1} = \mathbf{Y}_n + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$$

RK4 aplicado al sistema del disparo

Para

$$\begin{cases} y' = z, \\ z' = f(x, y, z), \end{cases}$$

se tiene

$$\mathbf{F}(x, y, z) = \begin{bmatrix} z \\ f(x, y, z) \end{bmatrix}.$$

El algoritmo del disparo tiene entonces dos niveles:

$$\boxed{\text{Disparo} \longrightarrow \text{PVI} \longrightarrow \text{RK4}}$$

El método RK4 realiza la integración espacial, mientras que el método del disparo modifica la condición inicial desconocida.

Idea clave

RK4 resuelve el PVI. El método del disparo determina qué condición inicial debe utilizarse.

Ejemplo 1: $y'' = y$

Example

Resolver

$$\begin{cases} y'' = y, \\ y(0) = 0, \\ y(1) = 2. \end{cases}$$

con

$$h = 0,1.$$

Definimos

$$z = y'$$

Entonces

$$\begin{cases} y' = z, \\ z' = y. \end{cases}$$

La pendiente inicial

$$z(0) = s$$

es desconocida.

Primer disparo

Elegimos

$$s_1 = 1,5.$$

Integrando mediante RK4 se obtiene

$$y(1; s_1) \approx 1,7628.$$

Segundo disparo

Elegimos ahora

$$s_2 = 2,5.$$

La integración mediante RK4 produce

$$y(1; s_2) \approx 2,9380.$$

La condición requerida es

$$\beta = 2.$$

Por tanto,

$$s_3 = s_1 + \frac{\beta - y(1; s_1)}{y(1; s_2) - y(1; s_1)}(s_2 - s_1).$$

Sustituyendo:

$$s_3 = 1,5 + \frac{2 - 1,7628}{2,9380 - 1,7628}(2,5 - 1,5) \approx 1,7018$$

Ejemplo 1: Resultado final

Utilizando

$$z(0) = s_3 = 1,7018,$$

se resuelve nuevamente

$$\begin{cases} y' = z, \\ z' = y, \end{cases}$$

con

$$y(0) = 0.$$

Se obtiene

$$y(1) \approx 2,0000$$

La solución exacta del problema es

$$y(x) = \frac{2}{\sinh(1)} \sinh(x).$$

Por tanto,

$$y'(0) = \frac{2}{\sinh(1)} \approx 1,701836.$$

El método numérico obtiene prácticamente el mismo valor.

Conclusión

El “disparo” correcto es

$$y'(0) \approx 1,7018$$

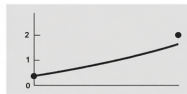
Cuadro: Resultados del método del disparo

Iteración	s	$y(1; s)$	$ y(1; s) - 2 $
1	1.5000	1.7628	0.2372
2	2.5000	2.9380	0.9380
3	1.7018	$\approx 2,0000$	≈ 0

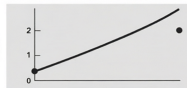
La interpolación lineal permite pasar de dos disparos iniciales a una pendiente muy próxima a la solución.

$$1,5 \longrightarrow 1,7018 \longrightarrow y(1) = 2$$

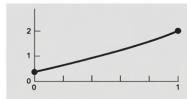
(a)



(b)



(c)



Python: Ejemplo 1 con solve_ivp

```
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt
```

```
a, b = 0.0, 1.0
alpha, beta = 0.0, 2.0
```

```
def f(x, Y):
    y, z = Y
    return [z, y]
```

```
def disparo(s):
    sol = solve_ivp(
        f, [a, b], [alpha, s],
        t_eval=np.linspace(a, b, 11),
        rtol=1e-10, atol=1e-12
    )
    return sol.y[0, -1], sol
```

```
s1 = 1.5
s2 = 2.5
```

```
Y1, sol1 = disparo(s1)
Y2, sol2 = disparo(s2)
```

```
s3 = s1 + (beta-Y1)/(Y2-Y1)*(s2-s1)
```

```
Y3, sol3 = disparo(s3)
```

```
print("s1 =", s1, " y(1) =", Y1)
print("s2 =", s2, " y(1) =", Y2)
print("s3 =", s3, " y(1) =", Y3)
```

```
plt.plot(sol1.t, sol1.y[0], '--', label='s1 = 1.5')
plt.plot(sol2.t, sol2.y[0], '--', label='s2 = 2.5')
plt.plot(sol3.t, sol3.y[0], label='disparo final')
plt.scatter([b], [beta], zorder=5)
```

```
plt.xlabel("x")
plt.ylabel("y(x)")
plt.grid()
plt.legend()
plt.show()
```

function dydx = ej(x,Y)
dydx = [Y(2); Y(1)];

x: variable independiente

Y: vector de EDOs

Y = [y(x); z(x)]

$$y' = z = \begin{bmatrix} y(2) \\ z' = y = \begin{bmatrix} y(1) \end{bmatrix}$$

>> [x,y] = ode45 (@ej,[0:0.1:1],[0,1.5]); >> [x,y] = ode45 (@ej,[0:0.1:1],[0,2.5]); >> [x,y] = ode45 (@ej,[0:0.1:1],[0,1.7018]);

	Y(1)	Y(2)
x	y	z
0	0	1.5000
0.1000	0.1503	1.5075
0.2000	0.3020	1.5301
0.3000	0.4568	1.5680
0.4000	0.6161	1.6216
0.5000	0.7816	1.6914
0.6000	0.9550	1.7782
0.7000	1.1379	1.8828
0.8000	1.3322	2.0062
0.9000	1.5398	2.1496
1.0000	1.7628	2.3146

	Y(1)	Y(2)
x	y	z
0	0	2.5000
0.1000	0.2504	2.5125
0.2000	0.5033	2.5502
0.3000	0.7613	2.6133
0.4000	1.0269	2.7027
0.5000	1.3027	2.8191
0.6000	1.5916	2.9637
0.7000	1.8965	3.1379
0.8000	2.2203	3.3436
0.9000	2.5663	3.5827
1.0000	2.9380	3.8577

	Y(1)	Y(2)
x	y	z
0	0	1.7018
0.1000	0.1705	1.7103
0.2000	0.3426	1.7359
0.3000	0.5182	1.7790
0.4000	0.6990	1.8398
0.5000	0.8868	1.9190
0.6000	1.0835	2.0174
0.7000	1.2910	2.1360
0.8000	1.5114	2.2760
0.9000	1.7469	2.4388
1.0000	2.0000	2.6260

Ejemplo 2: disparo lineal

Example

Resolver

$$\begin{cases} y'' = 3y + \cos(x), \\ y(0) = -0,3, \\ y\left(\frac{\pi}{2}\right) = -0,1 \end{cases}$$

Definimos

$$z = y'$$

Entonces

$$\begin{cases} y' = z, \\ z' = 3y + \cos(x) \end{cases}$$

Sea

$$\begin{aligned} a &= 0, & b &= \frac{\pi}{2}, \\ \alpha &= -0,3, & \beta &= -0,1. \end{aligned}$$

Ejemplo 2: soluciones auxiliares

Para aplicar el disparo lineal se construyen dos soluciones.

Solución base

$$y_1(0) = -0,3, \quad y_1'(0) = 0.$$

Se obtiene aproximadamente

$$y_1\left(\frac{\pi}{2}\right) \approx -0,2475.$$

Solución auxiliar

$$y_2(0) = 0, \quad y_2'(0) = 1.$$

Se obtiene

$$y_2\left(\frac{\pi}{2}\right) \approx 2,3010.$$

Ejemplo 2: cálculo de C

La solución buscada puede expresarse como

$$y(x) = y_1(x) + C y_2(x).$$

Imponemos la condición de frontera:

$$y\left(\frac{\pi}{2}\right) = -0,1.$$

Entonces

$$y_1\left(\frac{\pi}{2}\right) + C y_2\left(\frac{\pi}{2}\right) = -0,1.$$

Por tanto,

$$C = \frac{-0,1 - y_1(\pi/2)}{y_2(\pi/2)}.$$

Usando los valores proporcionados:

$$C = \frac{-0,1 - (-0,2475)}{2,3010} \approx 0,0641$$

Por consiguiente,

$$y(x) \approx y_1(x) + 0,0641 y_2(x)$$

Pendiente inicial corregida

La derivada de la solución es

$$y'(x) = y_1'(x) + C y_2'(x).$$

En $x = 0$:

$$y'(0) = y_1'(0) + C y_2'(0).$$

Como

$$y_1'(0) = 0, \quad y_2'(0) = 1,$$

se obtiene

$$y'(0) = C.$$

Por tanto,

$$y'(0) \approx 0,0641$$

y la solución final satisface aproximadamente

$$y(0) = -0,3, \quad y\left(\frac{\pi}{2}\right) = -0,1.$$

Python: Ejemplo 2

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

a = 0.0
b = np.pi/2

alpha = -0.3
beta = -0.1

def f(x, Y):
    y, z = Y
    return [z, 3*y + np.cos(x)]

x = np.linspace(a, b, 100)

# Solucion base
sol1 = solve_ivp(
    f, [a, b], [alpha, 0.0],
    t_eval=x, rtol=1e-10, atol=1e-12
)

# Solucion homogenea auxiliar
sol2 = solve_ivp(
    f, [a, b], [0.0, 1.0],
    t_eval=x, rtol=1e-10, atol=1e-12
)

Y1 = sol1.y[0, -1]
Y2 = sol2.y[0, -1]

C = (beta - Y1)/Y2

y = sol1.y[0] + C*sol2.y[0]
```

```
print("y1(b) =", Y1)
print("y2(b) =", Y2)
print("C =", C)
print("y(b) =", y[-1])

plt.plot(x, sol1.y[0], '--', label='y1')
plt.plot(x, sol2.y[0], '--', label='y2')
plt.plot(x, y, label='Solucion final')

plt.scatter([b], [beta], zorder=5)

plt.xlabel("x")
plt.ylabel("y(x)")
plt.grid()
plt.legend()
plt.show()
```

Observación sobre el disparo lineal

El procedimiento anterior es especialmente conveniente cuando la ecuación diferencial es **lineal**. Para una ecuación

$$y'' + p(x)y' + q(x)y = r(x)$$

la combinación

$$y(x) = y_1(x) + Cy_2(x)$$

permite calcular directamente la corrección necesaria.

Ventaja

Solo se requieren dos integraciones para determinar la pendiente corregida.

Para problemas no lineales, en cambio, puede ser necesario utilizar un procedimiento iterativo como:

bisección o secante/Newton

```
import numpy as np
from scipy.integrate import solve_ivp

def disparo_lineal(f, a, b, alpha, beta,
                  s1, s2, tol=1e-8, max_iter=50):

    def integrar(s):
        sol = solve_ivp(
            f,
            [a, b],
            [alpha, s],
            rtol=1e-10,
            atol=1e-12
        )
        return sol.y[0, -1], sol

    for k in range(max_iter):

        Y1, sol1 = integrar(s1)
        Y2, sol2 = integrar(s2)

        s3 = s1 + (beta-Y1)/(Y2-Y1)*(s2-s1)

        Y3, sol3 = integrar(s3)

        error = abs(Y3-beta)

        print(k+1, s3, Y3, error)

        if error < tol:
            return s3, sol3

        s1 = s2
        s2 = s3

    raise RuntimeError("El metodo no convergio.")
```

Aplicación 1: transferencia de calor

Una aplicación importante del método del disparo aparece en problemas estacionarios de transferencia de calor.

Consideremos

$$\frac{d^2 T}{dx^2} = 0,$$

con

$$T(0) = 100^\circ C, \quad T(L) = 20^\circ C.$$

Definimos

$$z = T'$$

Entonces

$$\begin{cases} T' = z, \\ z' = 0. \end{cases}$$

El gradiente inicial $z(0)$ es desconocido. El método del disparo determina el flujo térmico necesario para alcanzar

$$T(L) = 20^\circ C.$$

Python: aplicación de transferencia de calor

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
```

```
L = 1.0
T0 = 100.0
TL = 20.0
```

```
def f(x, Y):
    T, q = Y
    return [q, 0.0]

def integrar(s):
    return solve_ivp(
        f, [0, L], [T0, s],
        t_eval=np.linspace(0, L, 100),
        rtol=1e-10, atol=1e-12
    )

s1 = -50.0
s2 = -100.0

sol1 = integrar(s1)
sol2 = integrar(s2)

Y1 = sol1.y[0, -1]
Y2 = sol2.y[0, -1]

s3 = s1 + (TL-Y1)/(Y2-Y1)*(s2-s1)

sol3 = integrar(s3)

print("Pendiente inicial =", s3)
print("Temperatura final =", sol3.y[0, -1])

plt.plot(sol3.t, sol3.y[0])
plt.scatter([L], [TL], zorder=5)

plt.xlabel("x [m]")
plt.ylabel("T [oC]")
plt.grid()
plt.show()
```

Aplicación 2: deflexión de una viga

Otra aplicación del método del disparo corresponde a una viga modelada mediante una ecuación diferencial de cuarto orden.

Por ejemplo,

$$EI v''''(x) = q(x).$$

Para convertirla en un sistema de primer orden definimos

$$y_1 = v, \quad y_2 = v', \quad y_3 = v'', \quad y_4 = v'''. \quad$$

Entonces

$$y_1' = y_2,$$

$$y_2' = y_3,$$

$$y_3' = y_4,$$

$$y_4' = \frac{q(x)}{EI}.$$

El método del disparo permite determinar las condiciones iniciales desconocidas necesarias para satisfacer las condiciones en el extremo opuesto. **Python: aplicación a una viga**

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
```

```
L = 2.0
E = 200e9
I = 8.0e-6
q0 = 1000.0
```

```
def q(x):
    return q0

def beam(x, Y):
    v, vp, vpp, vppp = Y
    return [vp, vpp, vppp, q(x)/(E*I)]

# Viga simplemente apoyada:
# v(0)=0, v'(0)=s1
# v(L)=0, v'(L)=s2
#
# Para el ejemplo se determina el vector
# de pendientes mediante scipy.root.

from scipy.optimize import root

def residual(S):
    s1, s2 = S
    sol = solve_ivp( beam, [0, L], [0.0, s1, s2, 0.0],
                    rtol=1e-9, atol=1e-11
                    )
    return [sol.y[0, -1], sol.y[1, -1] ]

S0 = [0.0, 0.0]
resultado = root(residual, S0)
s1, s2 = resultado.x

sol = solve_ivp( beam, [0, L], [0.0, s1, s2, 0.0],
                t_eval=np.linspace(0, L, 200), rtol=1e-9, atol=1e-11
                )

print("v'(0) =", s1)
print("v''(0) =", s2)
print("v(L) =", sol.y[0, -1])
plt.plot(sol.t, sol.y[0])
plt.xlabel("x [m]")
plt.ylabel("Deflexion v(x) [m]")
plt.grid()
plt.show()
```

Método del disparo para problemas no lineales

Considérese

$$y'' = f(x, y, y').$$

La función de disparo es

$$F(s) = y(b; s) - \beta.$$

En un problema no lineal, $F(s)$ puede no ser lineal.

Se puede utilizar el método de la secante:

$$s_{k+1} = s_k - F(s_k) \frac{s_k - s_{k-1}}{F(s_k) - F(s_{k-1})}$$

donde

$$F(s_k) = y(b; s_k) - \beta.$$

El procedimiento continúa hasta

$$|F(s_k)| < \varepsilon.$$

Ventaja

No es necesario calcular explícitamente la derivada $F'(s)$.

Cuadro: Comparación de estrategias

Método	Convergencia	Robustez	Aplicación
Bisección	Lineal	Alta	No lineal
Secante	Superlineal	Media	No lineal
Newton	Cuadrática	Variable	No lineal
Disparo lineal	Directa	Alta	EDO lineal

Recomendación

Para problemas lineales:

disparo lineal

Para problemas no lineales:

disparo + secante/bisección/Newton

Problemas de convergencia

El método puede presentar dificultades cuando:

- las pendientes iniciales están muy alejadas de la solución;
- la función de disparo es altamente no lineal;
- pequeñas modificaciones en s producen grandes cambios en $y(b)$;
- existen varias soluciones;
- el problema es numéricamente inestable;
- el intervalo $[a, b]$ es muy grande.

Criterio de parada

Una condición frecuente es

$$|y(b; s_k) - \beta| < \varepsilon$$

y, adicionalmente,

$$|s_k - s_{k-1}| < \varepsilon_s.$$

Ventajas

- Conceptualmente sencillo.
- Convierte un PVF en PVI.
- Permite utilizar RK4 y otros integradores.
- Fácil de programar en Python.
- Adecuado para muchos problemas de ingeniería.

Desventajas

- Puede ser sensible a la pendiente inicial.
- Puede presentar inestabilidad.
- La convergencia no siempre está garantizada.
- Los problemas no lineales pueden requerir varias iteraciones.
- Puede existir más de una solución.

Resumen del método

El método del disparo sigue la siguiente estructura:

PVF $\longrightarrow$ PVI $\longrightarrow$ Integración $\longrightarrow$ Corrección

Para

$$y'' = f(x, y, y'),$$

se define

$$z = y'$$

y se obtiene

$$\begin{cases} y' = z, \\ z' = f(x, y, z). \end{cases}$$

La condición inicial desconocida

$$z(a) = s$$

se ajusta hasta obtener

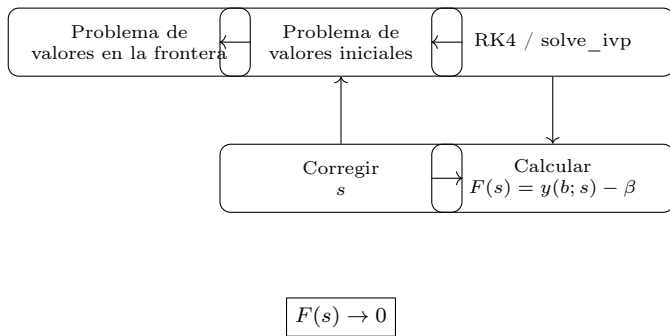
$$y(b) = \beta.$$

Idea central

El método del disparo convierte una condición de frontera desconocida en una variable que se ajusta iterativamente.

Conclusiones

1. El método del disparo permite resolver problemas de valores en la frontera mediante problemas de valores iniciales.
2. La pendiente inicial desconocida se convierte en una variable de ajuste.
3. La integración puede realizarse mediante RK4 o mediante integradores como `solve_ivp`.
4. Para problemas lineales, el disparo lineal permite calcular directamente la corrección mediante interpolación.
5. Para problemas no lineales pueden utilizarse bisección, secante o Newton.
6. El método tiene aplicaciones en transferencia de calor, mecánica estructural, dinámica de sistemas y otros problemas de ingeniería.



Ejercicio propuesto

Resolver mediante el método del disparo:

$$\begin{cases} y'' + 4y = 0, \\ y(0) = 1, \\ y\left(\frac{\pi}{4}\right) = 0. \end{cases}$$

Utilizar:

- $h = 0,05$;
- RK4;
- dos pendientes iniciales;
- disparo lineal;
- tolerancia $\varepsilon = 10^{-6}$.

Solicitar:

1. Tabla de iteraciones.
2. Pendiente inicial calculada.
3. Gráfica $y(x)$.
4. Comparación con la solución exacta.
5. Error absoluto.

Preguntas

Método del Disparo

Métodos Numéricos para Ingeniería

Método de las Diferencias Finitas

Método de las Diferencias Finitas

Idea fundamental

El Método de las Diferencias Finitas (DF) transforma una ecuación diferencial en un sistema algebraico mediante la aproximación de las derivadas por cocientes de diferencias.

Para una función $u(x)$ se define una malla:

$$x_i = x_0 + ih, \quad i = 0, 1, \dots, N,$$

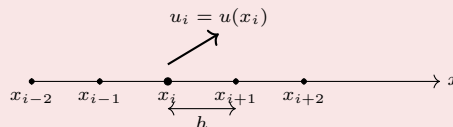
donde:

- h : tamaño de paso.
- x_i : puntos de la malla.
- $u_i = u(x_i)$: valor aproximado de la solución.

Objetivo

Convertir una ecuación diferencial continua en un sistema de ecuaciones algebraicas para los valores u_i .

Idea geométrica de la discretización



Malla uniforme

$$x_{i+1} - x_i = h$$

Forward - hacia adelante

$$x_i - x_{i-1} = h$$

Backward - hacia atrás

La discretización sustituye el dominio continuo por un conjunto finito de nodos.

Expansión de Taylor

Expansión alrededor del punto x_i :

$$u(x_i + h) = u_i + hu'_i + \frac{h^2}{2}u''_i + \frac{h^3}{6}u'''_i + \mathcal{O}(h^4)$$

y

$$u(x_i - h) = u_i - hu'_i + \frac{h^2}{2}u''_i - \frac{h^3}{6}u'''_i + \mathcal{O}(h^4)$$

Estas expresiones permiten construir aproximaciones numéricas para las derivadas.

Concepto importante

El término $\mathcal{O}(h^p)$ determina el orden de precisión del esquema numérico.

Diferencias hacia adelante (Forward),
partiendo de

$$u_{i+1} = u_i + hu'_i + \mathcal{O}(h^2),$$

se obtiene:

$$u'_i \approx \frac{u_{i+1} - u_i}{h} + \underbrace{\mathcal{O}(h)}_{\text{error}}$$

Diferencia hacia adelante (Forward)

$$D^+ u_i = \frac{u_{i+1} - u_i}{h}$$

Es especialmente útil en problemas donde se avanza en una dirección determinada.

diferencias hacia atrás (Backward),
utilizando

$$u_{i-1} = u_i - hu'_i + \mathcal{O}(h^2),$$

resulta:

$$u'_i \approx \frac{u_i - u_{i-1}}{h} + \underbrace{\mathcal{O}(h)}_{\text{error}}$$

Diferencia hacia atrás (Backward)

$$D^- u_i = \frac{u_i - u_{i-1}}{h}$$

Primera derivada y Segunda derivada

Diferencias centradas, restando las expansiones de Taylor:

$$u_{i+1} - u_{i-1} = 2hu'_i + \mathcal{O}(h^3).$$

Por tanto:

$$u'_i \approx \frac{u_{i+1} - u_{i-1}}{2h} + \underbrace{\mathcal{O}(h^2)}_{error}$$

Ventaja

La diferencia centrada presenta mayor precisión que las diferencias hacia adelante y hacia atrás para una misma malla uniforme.

Segunda derivada, sumando las expansiones de Taylor:

$$u_{i+1} + u_{i-1} = 2u_i + h^2u''_i + \mathcal{O}(h^4).$$

Por tanto:

$$u''_i \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} + \underbrace{\mathcal{O}(h^2)}_{error}$$

Esta expresión es una de las fórmulas fundamentales del Método de las Diferencias Finitas.

Problema general

Consideremos la EDO:

$$a(x)y'' + b(x)y' + c(x)y = d(x),$$

para: $a < x < b$.

Con condiciones de frontera:

$$y(a) = \alpha, \quad y(b) = \beta.$$

La malla:

$$x_i = a + ih, \quad h = \frac{b - a}{N}.$$

Definimos:

$$y_i \approx y(x_i).$$

Discretización de la EDO, utilizando diferencias centradas:

$$y'_i \approx \frac{y_{i+1} - y_{i-1}}{2h}$$

$$y''_i \approx \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2}$$

Sustituyendo:

$$a_i \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} + b_i \frac{y_{i+1} - y_{i-1}}{2h} + c_i y_i = d_i.$$

Agrupando:

$$A_i y_{i-1} + B_i y_i + C_i y_{i+1} = d_i,$$

donde:

$$A_i = \frac{a_i}{h^2} - \frac{b_i}{2h},$$

$$B_i = -\frac{2a_i}{h^2} + c_i,$$

$$C_i = \frac{a_i}{h^2} + \frac{b_i}{2h}$$

Formulación matricial Para los nodos interiores:

$$\begin{bmatrix} B_1 & C_1 & & & \\ A_2 & B_2 & C_2 & & \\ & \ddots & \ddots & \ddots & \\ & & A_{N-2} & B_{N-2} & C_{N-2} \\ & & & A_{N-1} & B_{N-1} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{N-2} \\ y_{N-1} \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ \vdots \\ d_{N-2} \\ d_{N-1} \end{bmatrix}.$$

Las condiciones de frontera modifican el vector del lado derecho.

Resultado: El problema diferencial se convierte en

$$\mathbf{A} \mathbf{y} = \mathbf{b}$$

y puede resolverse mediante métodos de álgebra lineal.

Algoritmo general del Método DF

1. Definir la ecuación diferencial.
2. Definir el dominio.
3. Especificar las condiciones de frontera.
4. Dividir el dominio en N intervalos.
5. Calcular:

$$h = \frac{b - a}{N}.$$

6. Construir los nodos:

$$x_i = a + ih.$$

7. Aproximar las derivadas mediante diferencias finitas.
8. Construir la matriz $\mathbf{A}$.
9. Construir el vector $\mathbf{b}$.
10. Resolver:
$$\mathbf{A}\mathbf{u} = \mathbf{b}.$$
11. Comparar con una solución analítica cuando exista.
12. Analizar el error y la convergencia.

Pseudocódigo

Algoritmo

1. Entrada:

$$a, b, N, \quad \alpha, \beta, \quad f(x, y, y').$$

2. Calcular:

$$h = (b - a)/N.$$

3. Generar:

$$x_i = a + ih.$$

4. Para $i = 1, \dots, N - 1$:

$$A_i y_{i-1} + B_i y_i + C_i y_{i+1} = d_i.$$

5. Incorporar las condiciones de frontera.

6. Resolver:

$$\mathbf{A}\mathbf{y} = \mathbf{b}.$$

7. Salida:

$$\{x_i, y_i\}.$$

Ejemplo 1: problema propuesto

Example

Resolver: $y''(x) = y(x)$
para: $0 < x < 1$,
con: $y(0) = 0, \quad y(1) = 2$.
Usaremos: $h = 0,1$.

La solución analítica es:

$$y(x) = \frac{2 \sinh(x)}{\sinh(1)}$$

y será utilizada para evaluar el error.

Para los primeros nodos:

$$y_0 - 2,01y_1 + y_2 = 0,$$

$$y_1 - 2,01y_2 + y_3 = 0,$$

$$y_2 - 2,01y_3 + y_4 = 0,$$

$$\vdots \quad \vdots$$

$$y_7 - 2,01y_8 + y_9 = 0$$

Las condiciones son:

$$y_0 = 0, \quad y_{10} = 2.$$

Por tanto:

$$\mathbf{A}\mathbf{y} = \mathbf{b}.$$

Discretización en cada nodo interior:

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} = y_i.$$

Por tanto:

$$y_{i+1} - 2y_i + y_{i-1} = h^2 y_i.$$

Agrupando:

$$y_{i-1} - (2 + h^2)y_i + y_{i+1} = 0$$

Para:

$$h = 0,1, \quad h^2 = 0,01,$$

obtenemos:

$$y_{i-1} - 2,01y_i + y_{i+1} = 0$$

Matriz para los nodos interiores:

$$\begin{bmatrix} -2,01 & 1 & & & & & & & & & \\ 1 & -2,01 & 1 & & & & & & & & \\ & 1 & -2,01 & 1 & & & & & & & \\ & & & \ddots & \ddots & \ddots & & & & & \\ & & & & 1 & -2,01 & 1 & & & & \\ & & & & & 1 & -2,01 & 1 & & & \\ & & & & & & 1 & -2,01 & 1 & & \\ & & & & & & & 1 & -2,01 & 1 & \\ & & & & & & & & 1 & -2,01 & 1 \end{bmatrix}$$

El término -2 aparece debido a la condición:

$$y_{10} = 2.$$

Ejemplo 1

$$\left. \begin{aligned} \frac{y(0,2) - 2y(0,1) + y(0)}{(0,1)^2} &= y(0,1) \\ \frac{y(0,3) - 2y(0,2) + y(0,1)}{(0,1)^2} &= y(0,2) \\ &\vdots = \vdots \\ \frac{y(1) - 2y(0,9) + y(0,8)}{(0,1)^2} &= y(0,9) \end{aligned} \right\} \Rightarrow \frac{1}{(0,1)^2} \begin{bmatrix} 1 & & & & \\ 1 & -2,01 & & & \\ & 1 & -2,01 & & \\ & & \ddots & \ddots & \\ & & & & 1 \end{bmatrix} \begin{bmatrix} y(0) \\ y(0,1) \\ y(0,2) \\ \vdots \\ y(1) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 2 \end{bmatrix}$$

x	y
0	0
0.1000	0.1705
0.2000	0.3427
0.3000	0.5183
0.4000	0.6991
0.5000	0.8869
0.6000	1.0836
0.7000	1.2911
0.8000	1.5115
0.9000	1.7470
1.0000	2.000

Ejemplo 1: código Python

```
import numpy as np
import matplotlib.pyplot as plt

# Dominio
a = 0.0
b = 1.0
N = 10

h = (b-a)/N
x = np.linspace(a, b, N+1)

# Numero de nodos interiores
n = N-1

# Matriz
A = np.zeros((n,n))

for i in range(n):
    A[i,i] = -2.0 - h**2

    if i > 0:
        A[i,i-1] = 1.0

    if i < n-1:
        A[i,i+1] = 1.0

# Condiciones de frontera
y0 = 0.0
yN = 2.0

rhs = np.zeros(n)
rhs[0] -= y0
rhs[-1] -= yN
```

Comparación y error

```
# Resolver
yi = np.linalg.solve(A, rhs)

y = np.zeros(N+1)
y[0] = y0
y[-1] = yN
y[1:-1] = yi

# Solucion exacta
y_exact = 2*np.sinh(x)/np.sinh(1)

# Error absoluto
error = np.abs(y-y_exact)

print(" x DF Exacta Error")
for xi, yi, ye, er in zip(x,y,y_exact,error):
    print(f"{xi:6.2f} {yi:12.6f} "
          f"{ye:12.6f} {er:12.4e}")

print("Error maximo =", np.max(error))

# Grafica
plt.plot(x, y, 'o-', label='Diferencias finitas')
plt.plot(x, y_exact, '--', label='Solucion exacta')
plt.xlabel('x')
plt.ylabel('y')
plt.grid()
plt.legend()
plt.show()
```

Ejemplo 2: ecuación de Poisson 1D

Example

Resolver:

$$-u''(x) = 1$$

para:

$$0 < x < 1,$$

con:

$$u(0) = 0, \quad u(1) = 0.$$

La solución exacta es:

$$u(x) = \frac{x(1-x)}{2}$$

Utilizaremos:

$$N = 10, \quad h = 0,1$$

Discretización de la ecuación es:

$$-u''(x) = 1.$$

Aplicando diferencias finitas:

$$-\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} = 1.$$

Multiplicando por h^2 :

$$-u_{i+1} + 2u_i - u_{i-1} = h^2.$$

Por tanto:

$$-u_{i-1} + 2u_i - u_{i+1} = h^2$$

Para:

$$h = 0,1 : \quad h^2 = 0,01.$$

Sistema Matricial el sistema para los nodos interiores es:

$$\begin{bmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_{N-2} \\ u_{N-1} \end{bmatrix} = h^2 \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \\ 1 \end{bmatrix}$$

Como:

$$u_0 = u_N = 0$$

no se agregan términos adicionales al vector derecho.

Ejemplo 2: código Python

```
import numpy as np
import matplotlib.pyplot as plt

a = 0.0
b = 1.0
N = 10

h = (b-a)/N
x = np.linspace(a,b,N+1)

n = N-1

# Matriz tridiagonal
A = np.zeros((n,n))

for i in range(n):
    A[i,i] = 2.0

    if i > 0:
        A[i,i-1] = -1.0

    if i < n-1:
        A[i,i+1] = -1.0

# Vector derecho
rhs = h**2*np.ones(n)

# Resolver
ui = np.linalg.solve(A,rhs)

u = np.zeros(N+1)
u[1:-1] = ui
```

```
# Solucion exacta
u_exact = x*(1-x)/2

error = np.abs(u-u_exact)

print("Error maximo =", np.max(error))

plt.plot(x,u,'o-',label='DF')
plt.plot(x,u_exact,'--',label='Exacta')
plt.xlabel('x')
plt.ylabel('u')
plt.grid()
plt.legend()
plt.show()
```

Error de discretización

Para la segunda derivada:

$$u''(x_i) = \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} + \mathcal{O}(h^2).$$

Por tanto, el método presenta un error de truncamiento:

$$E_T = \mathcal{O}(h^2)$$

para una malla uniforme y diferencias centradas.

Interpretación

Si aproximadamente se reduce:

$$h \rightarrow \frac{h}{2},$$

el error asociado al esquema debería disminuir aproximadamente:

$$E \rightarrow \frac{E}{4}.$$

Estudio de convergencia

Una forma práctica de verificar la convergencia consiste en utilizar:

$$N = 10, 20, 40, 80.$$

Calcular:

$$E_\infty = \max_i |u_i - u(x_i)|.$$

El orden experimental puede estimarse mediante:

$$p = \frac{\log(E_N / E_{2N})}{\log(2)}.$$

Para un esquema de segundo orden:

$$p \approx 2.$$

Aplicación 1: ecuación de calor 1D

Example

Consideremos:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

en:

$$0 < x < L.$$

Condiciones de frontera:

$$u(0, t) = T_L, \quad u(L, t) = T_R.$$

Condición inicial:

$$u(x, 0) = u_0(x).$$

Esta formulación permite estudiar la evolución temporal de la temperatura en una pared.

Esquema explícito: definimos

$$x_i = ih, \quad t^n = n\Delta t.$$

Aproximaciones:

$$\begin{aligned} \frac{\partial u}{\partial t} &\approx \frac{u_i^{n+1} - u_i^n}{\Delta t}, \\ \frac{\partial^2 u}{\partial x^2} &\approx \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{h^2}. \end{aligned}$$

Entonces:

$$u_i^{n+1} = u_i^n + r(u_{i+1}^n - 2u_i^n + u_{i-1}^n)$$

donde:

$$r = \frac{\alpha \Delta t}{h^2}.$$

Estabilidad:

Para el esquema explícito de la ecuación de calor:

$$r \leq \frac{1}{2}$$

es una condición clásica de estabilidad para la malla uniforme en una dimensión. Por tanto:

$$\Delta t \leq \frac{h^2}{2\alpha}.$$

Importante

Reducir h mejora la resolución espacial, pero también puede obligar a utilizar pasos temporales mucho menores.

Aplicación 1: código Python

```
import numpy as np
import matplotlib.pyplot as plt

# Parametros
L = 0.2
alpha = 5e-7

Nx = 40
dx = L/Nx

dt = 0.4*dx**2/alpha
Nt = 300

x = np.linspace(0,L,Nx+1)

# Temperatura inicial
T = 100*np.ones(Nx+1)

# Fronteras
TL = 20
TR = 20

T[0] = TL
T[-1] = TR

r = alpha*dt/dx**2

print("r =",r)
```

```
for n in range(Nt):

    Tnew = T.copy()

    for i in range(1,Nx):
        Tnew[i] = T[i] + r*(
            T[i+1] - 2*T[i] + T[i-1]
        )

    T = Tnew

plt.plot(x,T,'o-')
plt.xlabel('x [m]')
plt.ylabel('Temperatura [C]')
plt.grid()
plt.show()
```

Aplicación 2: ecuación de Laplace 2D

La ecuación de Laplace es:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0$$

La discretización mediante diferencias centrales produce:

$$\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h_x^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h_y^2} = 0.$$

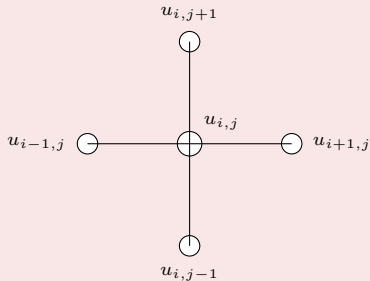
Para:

$$h_x = h_y = h,$$

se obtiene:

$$u_{i,j} = \frac{1}{4} (u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1})$$

Esténcil de cinco puntos



$$4u_{i,j} - u_{i+1,j} - u_{i-1,j} - u_{i,j+1} - u_{i,j-1} = 0$$

Este esquema recibe el nombre de **esténcil de cinco puntos**.

Aplicación 2: código Python

```
import numpy as np
import matplotlib.pyplot as plt

Nx = 40
Ny = 40

u = np.zeros((Ny,Nx))

# Condiciones de frontera
u[:,0] = 100
u[:,-1] = 0
u[0,:] = 0
u[-1,:] = 0

tol = 1e-6
max_iter = 10000

for k in range(max_iter):

    old = u.copy()

    for j in range(1,Ny-1):
        for i in range(1,Nx-1):

            u[j,i] = 0.25*(
                u[j,i+1] +
                u[j,i-1] +
                u[j+1,i] +
                u[j-1,i]
            )

    error = np.max(np.abs(u-old))

    if error < tol:
        print("Convergencia:", k)
        break
```

```
# Visualizacion
X,Y = np.meshgrid(
    np.arange(Nx),
    np.arange(Ny)
)

plt.figure()
plt.contourf(X,Y,u,30)
plt.colorbar(label='u')
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```

Aplicación adicional: distribución de temperatura

El mismo esquema puede utilizarse para:

- placas metálicas,
- paredes de edificios,
- disipadores térmicos,
- conductores eléctricos,
- problemas de conducción estacionaria.

Para conducción estacionaria sin generación:

$$\nabla^2 T = 0.$$

Para una placa rectangular:

$$T_{i,j} = \frac{1}{4} (T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1}).$$

El problema puede resolverse iterativamente mediante Jacobi, Gauss-Seidel¹⁷⁰ o SOR.

Aplicación adicional: método de Jacobi

```
import numpy as np

Nx, Ny = 30, 30
T = np.zeros((Ny,Nx))

# Temperaturas de frontera
T[:,0] = 100
T[:, -1] = 50
T[0,:] = 20
T[-1,:] = 20

tol = 1e-8

for iteration in range(10000):
    Tnew = T.copy()
    for j in range(1,Ny-1):
        for i in range(1,Nx-1):
            Tnew[j,i] = 0.25*(
                T[j,i+1] +
                T[j,i-1] +
                T[j+1,i] +
                T[j-1,i]
            )

    error = np.max(np.abs(Tnew-T))
    T = Tnew
    if error < tol:
        break

print("Iteraciones:", iteration)
print("Error:", error)
```

- Conceptualmente sencillo.
- Implementación computacional relativamente directa.
- Fácil de combinar con Python, MATLAB o C/C++.
- Muy eficiente para dominios regulares.
- Permite estudiar EDO y EDP.
- Las aproximaciones pueden obtenerse sistemáticamente mediante Taylor.
- Facilita el análisis del error y la convergencia.

Limitaciones

- Los dominios geoméricamente complejos pueden ser difíciles de representar.
- Las mallas no uniformes requieren formulaciones adicionales.
- Las condiciones de frontera deben incorporarse cuidadosamente.
- Algunos esquemas presentan restricciones de estabilidad.
- Refinar excesivamente la malla incrementa el costo computacional.
- La estabilidad y convergencia dependen del esquema utilizado.

Idea clave

Una discretización más fina no garantiza por sí sola una solución correcta: también deben verificarse consistencia, estabilidad y convergencia.

Ecuación diferencial $\longrightarrow$ Discretización $\longrightarrow \mathbf{Au} = \mathbf{b} \longrightarrow$ Solución numérica

Fórmulas fundamentales

Los pasos fundamentales son:

1. Definir la malla.
2. Aproximar las derivadas.
3. Construir las ecuaciones algebraicas.
4. Incorporar las condiciones de frontera.
5. Resolver el sistema.
6. Evaluar el error.
7. Verificar convergencia.

$$u'_i \approx \frac{u_{i+1} - u_{i-1}}{2h},$$

$$u''_i \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2},$$

$$u_i^{n+1} = u_i^n + r(u_{i+1}^n - 2u_i^n + u_{i-1}^n),$$

$$u_{i,j} = \frac{1}{4}(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1}).$$

La derivada continua se transforma en operaciones algebraicas.

Preguntas

Método de las Diferencias Finitas

Métodos Numéricos para Ingeniería

¿Preguntas?



Steven C. Chapra and Raymond P. Canale.
Métodos Numéricos para Ingenieros.
McGraw-Hill, 7th edition, 2015.